SQL

GUÍA INTERACTIVA

MySQL

Bases de datos relacionales de cero a intermedio: consultas, joins, ventanas, índices, transacciones y diseño, con resultados de un MySQL real.

Versión PDF · 30 de octubre de 2026
15 capítulos · Los ejemplos interactivos están en la app (npm run dev)

Índice

Empezá acá

1. Cómo usar esta guía

Parte 1 · Lo básico

2. Crear tablas
3. Insertar, actualizar y borrar
4. SELECT, WHERE y ORDER BY
5. Funciones y NULL
6. Agrupar: GROUP BY y HAVING
7. JOINs

Parte 2 · Intermedio

8. Subconsultas y CTE
9. Funciones de ventana
10. Índices y EXPLAIN
11. Transacciones y concurrencia
12. Diseño de tablas
13. Vistas, procedimientos, triggers y JSON
14. MySQL desde Node

Parte 3 · Repaso

15. Chuleta y ejercicios de entrevista

Empezá acá

La base de ejemplo, cómo se generan los resultados y cómo instalar MySQL.

Capítulo 1

Cómo usar esta guía

Qué es una base relacional, la base de ejemplo, el playground y MySQL en la Mac.

Esta guía va de cero a un nivel intermedio de **MySQL**: crear tablas, cargar y consultar datos, unir tablas, agrupar, y después subconsultas, funciones de ventana, índices, transacciones y diseño. Es lo que te van a preguntar en una entrevista para un puesto fullstack y lo que vas a usar desde un backend en Node.

Una base de datos relacional en un párrafo

Los datos viven en **tablas** (como hojas de cálculo con columnas de tipo fijo). Cada fila tiene un identificador único (la **clave primaria**) y las tablas se **relacionan** guardando el id de otra fila (una **clave foránea**). Con **SQL** describís *qué* datos querés, y el motor decide *cómo* buscarlos.

La base de ejemplo

Toda la guía usa la misma base: una **búsqueda laboral**. Empresas que publican ofertas, las tecnologías que pide cada oferta, tus postulaciones y las entrevistas de cada una.

```
empresas 1 ——— N ofertas N ——— N tecnologias        (a través de
  oferta_tecnologia)
                   |
                   1
```

```
                    |
                    1 postulaciones 1 ——— N entrevistas
```

src/demos/mysql/schema.sql

```sql
-- Base de datos de ejemplo de la guía de MySQL: una búsqueda laboral.
-- Este archivo crea las tablas (MySQL 8). Los datos están en datos.sql.

CREATE TABLE empresas (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  nombre     VARCHAR(100) NOT NULL UNIQUE,
  pais       CHAR(2)      NOT NULL,          -- código ISO: AR, PY, UY…
  empleados  INT          NULL CHECK (empleados > 0)
);

CREATE TABLE ofertas (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  empresa_id   INT          NOT NULL,
  titulo       VARCHAR(150) NOT NULL,
  seniority    ENUM('junior', 'semi', 'senior') NOT NULL,
  modalidad    ENUM('remoto', 'hibrido', 'presencial') NOT NULL DEFAULT
    'remoto',
  salario_min  INT  NULL,                    -- USD por mes; NULL si no
    lo publican
  salario_max  INT  NULL,
  publicada    DATE NOT NULL,
  FOREIGN KEY (empresa_id) REFERENCES empresas (id)
);

CREATE TABLE tecnologias (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  nombre  VARCHAR(50) NOT NULL UNIQUE
);

-- Tabla intermedia: una oferta pide muchas tecnologías y una tecnología
  aparece en muchas ofertas.
CREATE TABLE oferta_tecnologia (
  oferta_id      INT NOT NULL,
  tecnologia_id  INT NOT NULL,
  PRIMARY KEY (oferta_id, tecnologia_id),
  FOREIGN KEY (oferta_id) REFERENCES ofertas (id) ON DELETE CASCADE,
  FOREIGN KEY (tecnologia_id) REFERENCES tecnologias (id)
);

CREATE TABLE postulaciones (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  oferta_id  INT  NOT NULL UNIQUE,           -- una sola postulación por
    oferta
  estado     ENUM('postulado', 'entrevista', 'oferta', 'rechazado') NOT
    NULL DEFAULT 'postulado',
```

```
  fecha      DATE NOT NULL,
  notas      TEXT NULL,
  FOREIGN KEY (oferta_id) REFERENCES ofertas (id)
);

CREATE TABLE entrevistas (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  postulacion_id  INT      NOT NULL,
  fecha           DATETIME NOT NULL,
  tipo            ENUM('rrhh', 'tecnica', 'live coding', 'final') NOT
    NULL,
  resultado       ENUM('pendiente', 'paso', 'no paso') NOT NULL DEFAULT
    'pendiente',
  FOREIGN KEY (postulacion_id) REFERENCES postulaciones (id) ON DELETE
    CASCADE
);
```

Los datos están en `src/demos/mysql/datos.sql`: 10 empresas, 15 ofertas, 10 tecnologías, 8 postulaciones y 9 entrevistas. Pocos, a propósito: así podés verificar a mano cada resultado.

Los resultados son de MySQL de verdad

Cada consulta de la guía muestra abajo **lo que devolvió MySQL** al ejecutarla. No están escritos a mano: el script `npm run sql` corre todas las consultas contra un MySQL real (cada una sobre una base recién cargada) y guarda los resultados. Las consultas que **tienen** que fallar, para mostrar un error, también se comprueban: si un día dejaran de fallar, el script avisa.

SQL

```
SELECT nombre, pais, empleados
FROM empresas
ORDER BY empleados DESC
LIMIT 3;
```

RESULTADO EN MYSQL 8.0.46

nombre	pais	empleados
Delta Pay	BR	1500
Cumbre Data	MX	600
Pagos Sur	UY	300

3 filas

Probar en el navegador

MySQL no corre en un navegador, pero **SQLite** sí. Cuando una consulta da exactamente el mismo resultado en SQLite que en MySQL, aparece el botón ▶ **Probar en el navegador**: abre un editor con la consulta, la podés cambiar y ejecutar. Cada ejecución arranca con los datos originales, así que no rompés nada.

Las consultas que usan algo propio de MySQL (como `ENUM`, ciertas funciones de fecha o `EXPLAIN`) no tienen el botón: para esas, usá tu MySQL local.

Este es un playground libre para lo que quieras probar:

Instalar MySQL en la Mac

```
brew install mysql
brew services start mysql      # lo deja corriendo (y arranca solo al
  prender la Mac)
mysql -u root                  # entra a la consola, sin contraseña la
  primera vez
```

Para cargar la base de ejemplo, desde la carpeta del proyecto:

```
mysql -u root -e "CREATE DATABASE guia"
mysql -u root guia < src/demos/mysql/schema.sql
mysql -u root guia < src/demos/mysql/datos.sql
mysql -u root guia             # y a consultar
```

Y para regenerar los resultados de la guía: `npm run sql` (con `MYSQL_USER` y `MYSQL_PASSWORD` si no usás root sin contraseña).

💡 **Un cliente visual**

La consola alcanza, pero un cliente gráfico ayuda a explorar: **MySQL Workbench** (el oficial), **DBeaver** (gratis, sirve para cualquier base) o **TablePlus**. En VS Code también hay extensiones para conectarte y ejecutar SQL.

ℹ️ **Versiones**

Escrita en octubre de 2026. Los resultados se generaron con MySQL 8.0; todo lo de la guía funciona igual en MySQL 8.4 LTS (la versión que instala `brew` hoy).

Parte 1 · Lo básico

Crear tablas, cargar datos y consultarlos: lo que vas a escribir todos los días.

Capítulo 2

Crear tablas

CREATE TABLE, tipos de datos, restricciones y ALTER TABLE.

Antes de consultar datos hay que guardarlos, y antes de guardarlos hay que definir **dónde**: la base de datos y sus tablas. Esa parte de SQL se llama **DDL** (*Data Definition Language*): `CREATE`, `ALTER` y `DROP`.

Bases de datos

```
CREATE DATABASE guia CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE guia;            -- de acá en adelante, las tablas son de esta base
SHOW DATABASES;
SHOW TABLES;
```

> ℹ️ **utf8mb4**
>
> Es el juego de caracteres por defecto en MySQL 8 y el que hay que usar: soporta todo Unicode, incluidos emojis y la ñ. El viejo `utf8` (en realidad `utf8mb3`) no los soporta. La *collation* `utf8mb4_0900_ai_ci` compara sin distinguir mayúsculas ni acentos (`ai` = accent insensitive, `ci` = case insensitive): `'tereré' = 'TERERE'` da verdadero.

Crear una tabla

SQL

```sql
CREATE TABLE contactos (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  nombre     VARCHAR(100) NOT NULL,
  email      VARCHAR(255) NOT NULL UNIQUE,
  empresa_id INT NULL,
  creado     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (empresa_id) REFERENCES empresas (id)
);
DESCRIBE contactos;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	*NULL*	auto_increment
nombre	varchar(100)	NO		*NULL*	
email	varchar(255)	NO	UNI	*NULL*	
empresa_id	int	YES	MUL	*NULL*	
creado	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

5 filas

Cada columna tiene un **nombre**, un **tipo** y **restricciones** opcionales.

Los tipos que vas a usar

Tipo	Para	Ojo con
`INT` / `BIGINT`	Enteros, ids	`BIGINT` para ids de tablas enormes
`DECIMAL(10, 2)`	**Plata** y cantidades exactas	Nunca uses `FLOAT` / `DOUBLE` para plata: redondean
`VARCHAR(n)`	Textos cortos con largo máximo	`n` es en caracteres, no bytes
`TEXT`	Textos largos (notas, descripciones)	No puede tener `DEFAULT` y se indexa distinto
`CHAR(n)`	Textos de largo fijo (códigos de país)	Rellena con espacios

Tipo	Para	Ojo con
`DATE`	Fechas (`'2026-10-30'`)	Siempre en formato `AAAA-MM-DD`
`DATETIME`	Fecha y hora, sin zona horaria	Guarda lo que le das
`TIMESTAMP`	Fecha y hora en UTC	Convierte según la zona de la sesión; rango hasta 2038
`BOOLEAN`	Verdadero/falso	En realidad es `TINYINT(1)` : guarda 0 y 1
`ENUM('a', 'b')`	Un valor de una lista fija	Agregar opciones requiere `ALTER TABLE`
`JSON`	Datos semiestructurados	Útil, pero no reemplaza a las columnas

Restricciones

Restricción	Qué asegura
`PRIMARY KEY`	Identifica cada fila: única y no nula. Una por tabla (puede tener varias columnas).
`AUTO_INCREMENT`	MySQL asigna el siguiente número si no le pasás el id.
`NOT NULL`	La columna siempre tiene valor.
`UNIQUE`	No se repite (pero puede haber varios `NULL`).
`DEFAULT valor`	El valor si no se especifica uno.
`CHECK (condición)`	Una regla propia, como `empleados > 0` .
`FOREIGN KEY ... REFERENCES`	El valor tiene que existir en otra tabla (lo vemos en diseño).

Las restricciones son tu última línea de defensa: aunque el backend tenga un bug, la base no acepta datos inválidos.

SQL

```sql
INSERT INTO empresas (nombre, pais, empleados)
VALUES ('Nubelia', 'AR', 50);
```

RESULTADO EN MYSQL 8.0.46

```
ERROR 1062 (23000): Duplicate entry 'Nubelia' for key 'empresas.nombre'
```

Cambiar y borrar tablas

SQL

```sql
ALTER TABLE ofertas ADD COLUMN link VARCHAR(255) NULL AFTER titulo;
ALTER TABLE ofertas MODIFY COLUMN titulo VARCHAR(200) NOT NULL;
ALTER TABLE ofertas RENAME COLUMN link TO url;
SELECT id, titulo, url FROM ofertas LIMIT 2;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0
Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0
Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0
```

id	titulo	url
1	Frontend React Senior	*NULL*
2	Fullstack Node	*NULL*

2 filas

```sql
ALTER TABLE ofertas DROP COLUMN url;   -- borra la columna y sus datos
DROP TABLE contactos;                  -- borra la tabla entera
DROP TABLE IF EXISTS contactos;        -- sin error si no existe
```

> ⚠️ **DDL no se deshace**
>
> En MySQL, `CREATE`, `ALTER` y `DROP` hacen un *commit* implícito: no se pueden deshacer con `ROLLBACK`, aunque estés dentro de una transacción. En producción, los cambios de estructura se hacen con **migraciones** versionadas (Prisma, Drizzle, Knex, Flyway…), nunca a mano.

🎤 **En una entrevista**

"¿Qué tipo usarías para guardar un precio?" → "DECIMAL, por ejemplo DECIMAL(10, 2). FLOAT y DOUBLE son de punto flotante binario y no representan exacto valores como 0.1, así que los totales terminan con errores de redondeo. Otra opción es guardar centavos en un entero."

Capítulo 3

Insertar, actualizar y borrar

INSERT, UPDATE y DELETE, las claves foráneas en acción y el upsert.

La parte de SQL que cambia datos se llama **DML** (*Data Manipulation Language*): `INSERT`, `UPDATE` y `DELETE`. Junto con `SELECT`, es el famoso **CRUD**: *Create, Read, Update, Delete*.

INSERT

SQL

```sql
INSERT INTO empresas (nombre, pais, empleados)
VALUES ('Lapacho Soft', 'PY', 40);

INSERT INTO tecnologias (nombre)
VALUES ('Docker'), ('GraphQL'), ('Tailwind');

SELECT * FROM empresas WHERE pais = 'PY';
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 1 fila afectada
Query OK, 3 filas afectadas · Records: 3 Duplicates: 0 Warnings: 0
```

id	nombre	pais	empleados
5	Tereré Tech	PY	25
6	Yerba Labs	PY	12
10	Quebracho AI	PY	8
11	Lapacho Soft	PY	40

4 filas

- Escribí siempre la **lista de columnas**: si mañana alguien agrega una columna, tu `INSERT` sigue funcionando.

- Las que no nombrás toman su `DEFAULT` (o `NULL`). El `id` lo pone `AUTO_INCREMENT`.
- Varias filas en un solo `INSERT` es mucho más rápido que muchos `INSERT` separados.

Si una fila viola una restricción, MySQL la rechaza:

SQL

```sql
INSERT INTO ofertas (empresa_id, titulo, seniority, publicada)
VALUES (99, 'Frontend', 'senior', '2026-10-30');
```

RESULTADO EN MYSQL 8.0.46

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key
constraint fails (`guia_sql`.`ofertas`, CONSTRAINT `ofertas_ibfk_1`
FOREIGN KEY (`empresa_id`) REFERENCES `empresas` (`id`))
```

UPDATE

SQL

```sql
UPDATE postulaciones
SET estado = 'entrevista', notas = 'Me escribieron por mail'
WHERE id = 4;

SELECT id, estado, notas FROM postulaciones WHERE id = 4;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 1 fila afectada · Rows matched: 1 Changed: 1 Warnings: 0
```

id	estado	notas
4	entrevista	Me escribieron por mail

1 fila

> ⚠️ **Siempre con WHERE**
>
> Un `UPDATE` o un `DELETE` **sin** `WHERE` cambia o borra **todas** las filas. Antes de ejecutarlo, probá el mismo `WHERE` con un `SELECT` para ver qué filas toca. En la consola, `mysql --safe-updates` rechaza los `UPDATE` / `DELETE` sin un `WHERE` por clave.

Se puede actualizar a partir del valor actual:

SQL

```sql
UPDATE ofertas
SET salario_min = salario_min + 200,
    salario_max = salario_max + 200
WHERE empresa_id = 6;

SELECT titulo, salario_min, salario_max FROM ofertas WHERE empresa_id =
  6;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 1 fila afectada · Rows matched: 1 Changed: 1 Warnings: 0
```

titulo	salario_min	salario_max
Frontend React + TypeScript	3000	3500

1 fila

DELETE

SQL

```sql
DELETE FROM entrevistas WHERE resultado = 'pendiente';
SELECT COUNT(*) AS quedan FROM entrevistas;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 3 filas afectadas
```

quedan
6

1 fila

Si otra tabla apunta a la fila con una clave foránea, MySQL no te deja borrarla (a menos que la relación tenga `ON DELETE CASCADE`):

SQL

```sql
DELETE FROM empresas WHERE id = 1;
```

RESULTADO EN MYSQL 8.0.46

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key
constraint fails (`guia_sql`.`ofertas`, CONSTRAINT `ofertas_ibfk_1`
FOREIGN KEY (`empresa_id`) REFERENCES `empresas` (`id`))
```

Comando	Qué hace
`DELETE FROM t WHERE ...`	Borra las filas que cumplen la condición. Se puede deshacer en una transacción.
`DELETE FROM t`	Borra todas, fila por fila.
`TRUNCATE TABLE t`	Vacía la tabla de golpe y reinicia el `AUTO_INCREMENT`. Es DDL: no se deshace.

Insertar o actualizar (upsert)

Un caso muy común: "si existe, actualizalo; si no, crealo". MySQL lo resuelve con `ON DUPLICATE KEY UPDATE`, que se activa cuando el `INSERT` choca con una `PRIMARY KEY` o un `UNIQUE`:

SQL

```sql
INSERT INTO empresas (nombre, pais, empleados)
VALUES ('Nubelia', 'AR', 150), ('Lapacho Soft', 'PY', 40) AS nueva
ON DUPLICATE KEY UPDATE empleados = nueva.empleados;

SELECT nombre, empleados FROM empresas WHERE nombre IN ('Nubelia',
  'Lapacho Soft');
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 3 filas afectadas · Records: 2 Duplicates: 1 Warnings: 0
```

nombre	empleados
Lapacho Soft	40

nombre	empleados
Nubelia	150

2 filas

Nubelia ya existía (el nombre es `UNIQUE`), así que se actualizó; Lapacho Soft no, así que se insertó. `AS nueva` le pone un nombre a la fila que se intentó insertar, para usar sus valores en el `UPDATE`. Fijate en el resultado: MySQL cuenta 2 filas afectadas por cada actualización y 1 por cada inserción.

> **ℹ️ La forma vieja: VALUES()**
>
> En código anterior a MySQL 8.0.19 vas a ver `ON DUPLICATE KEY UPDATE empleados = VALUES(empleados)`. Hace lo mismo, pero está deprecado (MySQL devuelve un *warning*). Otra alternativa, `REPLACE INTO`, borra la fila vieja e inserta una nueva: cambia el id y dispara las claves foráneas, así que casi nunca es lo que querés.

> **🎤 En una entrevista**
>
> "¿Qué diferencia hay entre DELETE y TRUNCATE?" → "DELETE borra fila por fila, acepta WHERE, dispara triggers y se puede deshacer en una transacción. TRUNCATE vacía la tabla entera de una, es mucho más rápido, reinicia el AUTO_INCREMENT y es DDL, así que no se puede deshacer."

Capítulo 4

SELECT, WHERE y ORDER BY

Elegir columnas, filtrar, ordenar, paginar y DISTINCT.

`SELECT` es el 90% del SQL que vas a escribir. La estructura completa tiene este orden, y cada parte es opcional salvo `SELECT`:

```
SELECT    columnas
FROM      tabla
WHERE     condición sobre cada fila
GROUP BY  columnas para agrupar
HAVING    condición sobre cada grupo
ORDER BY  orden
LIMIT     cuántas filas
```

En este capítulo: `SELECT`, `WHERE`, `ORDER BY` y `LIMIT`. Agrupar viene más adelante.

Elegir columnas

SQL

```sql
SELECT titulo, salario_min, salario_max,
       salario_max - salario_min AS rango
FROM ofertas
WHERE empresa_id = 1;
```

RESULTADO EN MYSQL 8.0.46

titulo	salario_min	salario_max	rango
Frontend React Senior	3000	4000	1000
Fullstack Node	2200	3000	800

2 filas

- `SELECT *` trae todas las columnas. Está bien para explorar, pero en el código de una app pedí solo las que usás.
- `AS` le pone un **alias** a una columna (o a una expresión). El `AS` es opcional, pero se lee mejor.

Filtrar con WHERE

SQL

```sql
SELECT titulo, seniority, modalidad, salario_max
FROM ofertas
WHERE modalidad = 'remoto'
  AND seniority = 'senior'
  AND salario_max >= 4000;
```

RESULTADO EN MYSQL 8.0.46

titulo	seniority	modalidad	salario_max
Frontend React Senior	senior	remoto	4000
Frontend Engineer	senior	remoto	4500
Tech Lead Frontend	senior	remoto	5500
Senior Frontend Engineer	senior	remoto	6000

4 filas

Operador	Ejemplo
`=`, `<>` (o `!=`), `<`, `>`, `<=`, `>=`	`salario_max >= 4000`
`AND`, `OR`, `NOT`	`modalidad = 'remoto' AND NOT seniority = 'junior'`
`BETWEEN a AND b`	`publicada BETWEEN '2026-10-01' AND '2026-10-31'` (incluye los extremos)
`IN (...)`	`pais IN ('AR', 'PY', 'UY')`
`LIKE`	`titulo LIKE '%React%'` (`%` = cualquier texto, `_` = un carácter)
`IS NULL`, `IS NOT NULL`	`salario_max IS NULL`

SQL

```sql
SELECT nombre, pais
```

```sql
FROM empresas
WHERE pais IN ('AR', 'PY', 'UY')
  AND nombre LIKE '%a%'
ORDER BY pais, nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	pais
Mate Cloud	AR
Nubelia	AR
Quebracho AI	PY
Yerba Labs	PY
Pagos Sur	UY

5 filas

> ⚠️ **AND va antes que OR**
>
> `WHERE pais = 'AR' OR pais = 'PY' AND empleados > 50` se lee como `pais = 'AR' OR (pais = 'PY' AND empleados > 50)`. Cuando mezcles, usá paréntesis siempre.

Ordenar y limitar

SQL

```sql
SELECT titulo, salario_max
FROM ofertas
WHERE salario_max IS NOT NULL
ORDER BY salario_max DESC, titulo
LIMIT 5;
```

RESULTADO EN MYSQL 8.0.46

titulo	salario_max
Senior Frontend Engineer	6000
Tech Lead Frontend	5500
Frontend Engineer	4500
Data Engineer	4200
Frontend React Senior	4000

5 filas

- `ORDER BY` ordena por una o más columnas. `ASC` (por defecto) o `DESC`.
- **Sin `ORDER BY` no hay orden garantizado**: MySQL puede devolver las filas en cualquier orden, y ese orden puede cambiar.
- `LIMIT n` devuelve las primeras `n`. Con `LIMIT n OFFSET m` saltea `m` filas (paginación).

SQL

```sql
SELECT id, titulo
FROM ofertas
ORDER BY id
LIMIT 3 OFFSET 6;
```

RESULTADO EN MYSQL 8.0.46

id	titulo
7	Tech Lead Frontend
8	Fullstack Next.js
9	Frontend React + TypeScript

3 filas

> 💡 **Paginación con OFFSET en tablas grandes**
>
> `OFFSET 100000` obliga a MySQL a leer y descartar 100.000 filas. Para listas largas (un feed, un scroll infinito) se usa **paginación por cursor**: `WHERE id > :ultimoIdVisto ORDER BY id LIMIT 20`, que con un índice es igual de rápida en cualquier página.

Valores distintos

SQL

```sql
SELECT DISTINCT seniority, modalidad
FROM ofertas
ORDER BY seniority, modalidad;
```

RESULTADO EN MYSQL 8.0.46

seniority	modalidad
junior	remoto
junior	presencial
semi	remoto
semi	hibrido
senior	remoto
senior	hibrido
senior	presencial

7 filas

`DISTINCT` elimina las filas repetidas **considerando todas las columnas elegidas**.

⚠️ Los ENUM se ordenan por posición

Fijate el orden: `junior`, `semi`, `senior` y `remoto`, `hibrido`, `presencial`. No es alfabético: una columna `ENUM` se ordena por la **posición** de cada valor en la definición de la tabla. Es una particularidad de MySQL (en otras bases, o en el playground del navegador, el orden sería alfabético).

🎤 En una entrevista

"¿En qué orden se ejecuta un SELECT?" → "Aunque se escribe SELECT primero, se evalúa FROM (y los JOIN), después WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY y al final LIMIT. Por eso en el WHERE no puedo usar un alias del SELECT, pero en el ORDER BY sí."

Capítulo 5

Funciones y NULL

Texto, números, fechas, CASE y cómo se comporta NULL.

En el `SELECT` y en el `WHERE` podés usar funciones para transformar valores, y `CASE` para escribir condiciones. Y hay que entender bien `NULL`, que es la fuente de bugs más común en SQL.

Texto

SQL

```sql
SELECT nombre,
       UPPER(nombre) AS mayusculas,
       LENGTH(nombre) AS bytes,
       CHAR_LENGTH(nombre) AS letras,
       CONCAT(nombre, ' (', pais, ')') AS etiqueta,
       SUBSTRING(nombre, 1, 4) AS inicio
FROM empresas
WHERE pais = 'PY';
```

RESULTADO EN MYSQL 8.0.46

nombre	mayusculas	bytes	letras	etiqueta	inicio
Tereré Tech	TERERÉ TECH	12	11	Tereré Tech (PY)	Tere
Yerba Labs	YERBA LABS	10	10	Yerba Labs (PY)	Yerb
Quebracho AI	QUEBRACHO AI	12	12	Quebracho AI (PY)	Queb

3 filas

Fijate en Tereré Tech: `LENGTH` cuenta **bytes** (la `é` ocupa 2 en UTF-8) y `CHAR_LENGTH` cuenta **caracteres**. Otras útiles: `LOWER`, `TRIM`, `REPLACE(texto, 'a', 'b')`, `LEFT` / `RIGHT`, `LPAD`, `CONCAT_WS(', ', a, b)` (concatena con separador e ignora los `NULL`).

Números

SQL

```sql
SELECT titulo,
       (salario_min + salario_max) / 2 AS promedio,
       ROUND((salario_min + salario_max) / 2 / 1000, 1) AS miles,
       salario_max * 12 AS anual,
       MOD(salario_max, 1000) AS resto
FROM ofertas
WHERE empresa_id IN (1, 6);
```

RESULTADO EN MYSQL 8.0.46

titulo	promedio	miles	anual	resto
Frontend React Senior	3500.0000	3.5	48000	0
Fullstack Node	2600.0000	2.6	36000	0
Frontend React + TypeScript	3050.0000	3.1	39600	300

3 filas

La división `/` en MySQL siempre da decimal (`3500.0000`). Para división entera, `DIV`. También: `ROUND`, `CEIL`, `FLOOR`, `ABS`, `GREATEST(a, b, ...)`, `LEAST(...)`.

Fechas

SQL

```sql
SELECT titulo, publicada,
       YEAR(publicada) AS anio,
       MONTHNAME(publicada) AS mes,
       DATE_FORMAT(publicada, '%d/%m/%Y') AS fecha_ar,
       DATEDIFF('2026-10-30', publicada) AS dias_publicada,
       DATE_ADD(publicada, INTERVAL 30 DAY) AS vence
FROM ofertas
WHERE empresa_id = 2;
```

RESULTADO EN MYSQL 8.0.46

```
*** fila 1 ***
titulo       Frontend Engineer
publicada    2026-10-01
```

anio	2026
mes	October
fecha_ar	01/10/2026
dias_publicada	29
vence	2026-10-31
*** fila 2 ***	
titulo	React Native Developer
publicada	2026-10-05
anio	2026
mes	October
fecha_ar	05/10/2026
dias_publicada	25
vence	2026-11-04

Función	Qué hace
`NOW()`, `CURDATE()`	Fecha y hora actual, fecha actual
`DATE(datetime)`	Se queda con la fecha
`YEAR()`, `MONTH()`, `DAY()`, `DAYOFWEEK()`	Partes de una fecha
`DATE_ADD(f, INTERVAL n DAY)`, `DATE_SUB(...)`	Sumar o restar (`DAY`, `WEEK`, `MONTH`, `HOUR` ...)
`DATEDIFF(a, b)`	Días entre dos fechas
`TIMESTAMPDIFF(MONTH, a, b)`	Diferencia en la unidad que digas
`DATE_FORMAT(f, '%d/%m/%Y')`	Formatear

💡 Filtrar por fechas

Para "las de octubre", preferí un rango: `publicada >= '2026-10-01' AND publicada < '2026-11-01'`. Con `MONTH(publicada) = 10` el resultado es el mismo, pero MySQL no puede usar un índice sobre `publicada` porque tiene que calcular la función fila por fila (lo vemos en índices).

CASE: un if dentro de la consulta

SQL

```sql
SELECT titulo, salario_max,
       CASE
         WHEN salario_max IS NULL THEN 'sin publicar'
         WHEN salario_max >= 4000 THEN 'alto'
         WHEN salario_max >= 2500 THEN 'medio'
         ELSE 'bajo'
       END AS rango
FROM ofertas
ORDER BY salario_max DESC;
```

RESULTADO EN MYSQL 8.0.46

titulo	salario_max	rango
Senior Frontend Engineer	6000	alto
Tech Lead Frontend	5500	alto
Frontend Engineer	4500	alto
Data Engineer	4200	alto
Frontend React Senior	4000	alto
React Developer (LatAm)	3500	medio
Frontend Senior	3400	medio
Frontend React + TypeScript	3300	medio
React Native Developer	3200	medio
Fullstack Node	3000	medio
Fullstack Next.js	2800	medio
Junior Frontend	2000	bajo
Frontend Junior	1600	bajo
Desarrollador React	*NULL*	sin publicar
Backend Node.js	*NULL*	sin publicar

15 filas

`CASE` evalúa las condiciones en orden y devuelve la primera que se cumple. Sirve en el `SELECT`, en el `ORDER BY` y adentro de funciones de agregación (lo vemos en el capítulo siguiente). Para casos de dos opciones, MySQL tiene `IF(condición, si, no)`.

NULL: "no sé"

`NULL` no es cero ni texto vacío: significa **valor desconocido** o ausente. Y cualquier comparación con algo desconocido da... desconocido.

SQL

```sql
SELECT NULL = NULL AS igual,
       NULL <> 1 AS distinto,
       NULL + 100 AS suma,
       NULL IS NULL AS es_null,
       COALESCE(NULL, NULL, 'primero no nulo') AS coalesce_;
```

RESULTADO EN MYSQL 8.0.46

igual	distinto	suma	es_null	coalesce_
NULL	*NULL*	*NULL*	1	primero no nulo

1 fila

Eso tiene una consecuencia que sorprende a todos:

SQL

```sql
SELECT COUNT(*) AS total,
       SUM(salario_max > 3000) AS mas_de_3000,
       SUM(salario_max <= 3000) AS hasta_3000,
       SUM(salario_max IS NULL) AS sin_salario
FROM ofertas;
```

RESULTADO EN MYSQL 8.0.46

total	mas_de_3000	hasta_3000	sin_salario
15	9	4	2

1 fila

Las dos ofertas sin salario **no están en ninguno de los dos grupos**: `NULL > 3000` no es verdadero ni falso. Un `WHERE salario_max <= 3000` tampoco las trae. (Este truco de `SUM(condición)` funciona porque en MySQL una condición vale 1 o 0.)

Para...	Usá	No
Preguntar si es nulo	`col IS NULL` / `IS NOT NULL`	`col = NULL` (siempre da `NULL`)
Reemplazar un nulo	`COALESCE(col, 'valor')` o `IFNULL(col, 'valor')`	
Comparar contando nulos como iguales	`a <=> b` (operador de MySQL)	`a = b`

SQL

```sql
SELECT titulo, COALESCE(salario_max, 0) AS salario,
  IFNULL(CAST(salario_max AS CHAR), 'a convenir') AS texto
FROM ofertas
WHERE empresa_id IN (3, 8);
```

RESULTADO EN MYSQL 8.0.46

titulo	salario	texto
Desarrollador React	0	a convenir
Senior Frontend Engineer	6000	6000
Backend Node.js	0	a convenir

3 filas

🎤 En una entrevista

"¿Por qué WHERE col = NULL no trae nada?" → "Porque NULL significa desconocido y cualquier comparación con NULL da NULL, no verdadero. Hay que usar IS NULL o IS NOT NULL. Por lo mismo, un NOT IN con una subconsulta que tiene un NULL no devuelve filas."

Capítulo 6

Agrupar: GROUP BY y HAVING

COUNT, SUM y AVG, GROUP BY, HAVING, agregaciones condicionales y GROUP_CONCAT.

Las **funciones de agregación** resumen muchas filas en un valor: cuántas hay, cuánto suman, el promedio. Con `GROUP BY`, ese resumen se hace **por grupo**.

Agregaciones sobre toda la tabla

SQL

```sql
SELECT COUNT(*) AS ofertas,
       COUNT(salario_max) AS con_salario,
       MIN(salario_min) AS minimo,
       MAX(salario_max) AS maximo,
       AVG(salario_max) AS promedio_max,
       SUM(salario_max) AS suma
FROM ofertas;
```

RESULTADO EN MYSQL 8.0.46

ofertas	con_salario	minimo	maximo	promedio_max	suma
15	13	1200	6000	3615.3846	47000

1 fila

- `COUNT(*)` cuenta filas. `COUNT(columna)` cuenta los valores **no nulos**: por eso da 13 y no 15.
- `SUM`, `AVG`, `MIN` y `MAX` también **ignoran los** `NULL`. El promedio es sobre las 13 ofertas con salario.
- `COUNT(DISTINCT columna)` cuenta valores distintos.

GROUP BY

SQL

```sql
SELECT seniority,
       COUNT(*) AS ofertas,
       ROUND(AVG(salario_max)) AS promedio_max
FROM ofertas
GROUP BY seniority
ORDER BY promedio_max DESC;
```

RESULTADO EN MYSQL 8.0.46

seniority	ofertas	promedio_max
senior	7	4414
semi	6	3125
junior	2	1800

3 filas

`GROUP BY seniority` arma un grupo por cada valor distinto y calcula las agregaciones adentro de cada uno. Se puede agrupar por varias columnas:

SQL

```sql
SELECT seniority, modalidad, COUNT(*) AS ofertas
FROM ofertas
GROUP BY seniority, modalidad
ORDER BY seniority, ofertas DESC;
```

RESULTADO EN MYSQL 8.0.46

seniority	modalidad	ofertas
junior	presencial	1
junior	remoto	1
semi	remoto	5
semi	hibrido	1
senior	remoto	5
senior	hibrido	1
senior	presencial	1

7 filas

⚠️ ONLY_FULL_GROUP_BY

En un `SELECT` con `GROUP BY`, cada columna tiene que estar en el `GROUP BY` o adentro de una agregación. Si no, ¿qué `titulo` mostraría para el grupo `senior`, si hay seis? MySQL 8 da error (versiones viejas devolvían uno cualquiera, y mucho código legacy depende de eso):

SQL

```sql
SELECT seniority, titulo, COUNT(*)
FROM ofertas
GROUP BY seniority;
```

RESULTADO EN MYSQL 8.0.46

```
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP
BY clause and contains nonaggregated column
'guia_sql.ofertas.titulo' which is not functionally dependent on
columns in GROUP BY clause; this is incompatible with
sql_mode=only_full_group_by
```

HAVING: filtrar grupos

`WHERE` filtra **filas** antes de agrupar; `HAVING` filtra **grupos** después de agregar.

SQL

```sql
SELECT empresa_id,
       COUNT(*) AS ofertas,
       MAX(salario_max) AS mejor
FROM ofertas
WHERE modalidad = 'remoto'
GROUP BY empresa_id
HAVING COUNT(*) >= 2
ORDER BY mejor DESC;
```

RESULTADO EN MYSQL 8.0.46

empresa_id	ofertas	mejor
8	2	6000
1	2	4000
9	2	3500

3 filas

Leído en orden: de las ofertas **remotas** (`WHERE`), agrupá por empresa, y quedate con las empresas que tienen **al menos 2** (`HAVING`).

Agregaciones condicionales

Con `CASE` (o una condición) adentro de una agregación se arma una tabla de resumen en una sola pasada:

SQL

```sql
SELECT estado,
       COUNT(*) AS postulaciones,
       MIN(fecha) AS primera,
       MAX(fecha) AS ultima
FROM postulaciones
GROUP BY estado
ORDER BY postulaciones DESC;
```

RESULTADO EN MYSQL 8.0.46

estado	postulaciones	primera	ultima
entrevista	3	2026-09-05	2026-10-19
rechazado	2	2026-09-16	2026-10-10
postulado	2	2026-10-13	2026-10-21
oferta	1	2026-10-02	2026-10-02

4 filas

SQL

```sql
SELECT COUNT(*) AS total,
       SUM(CASE WHEN modalidad = 'remoto' THEN 1 ELSE 0 END) AS remotas,
```

```sql
       SUM(CASE WHEN modalidad <> 'remoto' THEN 1 ELSE 0 END) AS
         no_remotas,
       ROUND(100 * SUM(CASE WHEN modalidad = 'remoto' THEN 1 ELSE 0 END)
         / COUNT(*)) AS porcentaje_remoto
FROM ofertas;
```

RESULTADO EN MYSQL 8.0.46

total	remotas	no_remotas	porcentaje_remoto
15	11	4	73

1 fila

GROUP_CONCAT

Una función muy útil de MySQL: junta los valores de un grupo en un string.

SQL

```sql
SELECT pais,
       COUNT(*) AS empresas,
       GROUP_CONCAT(nombre ORDER BY nombre SEPARATOR ', ') AS nombres
FROM empresas
GROUP BY pais
ORDER BY empresas DESC, pais;
```

RESULTADO EN MYSQL 8.0.46

pais	empresas	nombres
PY	3	Quebracho AI, Tereré Tech, Yerba Labs
AR	2	Mate Cloud, Nubelia
BR	1	Delta Pay
CL	1	Andes Labs
MX	1	Cumbre Data
US	1	Remoto.dev
UY	1	Pagos Sur

7 filas

🎤 En una entrevista

"¿Qué diferencia hay entre WHERE y HAVING?" → "WHERE filtra filas antes de agrupar y no puede usar agregaciones. HAVING filtra los grupos después del GROUP BY y sí puede usar COUNT, SUM, etc. Si una condición se puede poner en el WHERE, conviene ponerla ahí: se descartan filas antes y se agrupa menos."

Capítulo 7

JOINs

INNER y LEFT JOIN, buscar lo que falta, ON vs WHERE, muchos a muchos y UNION.

Los datos están repartidos en tablas: una oferta guarda `empresa_id`, no el nombre de la empresa. Un **JOIN** junta filas de dos tablas que se corresponden, para leerlas como si fueran una sola.

INNER JOIN

SQL

```sql
SELECT o.titulo, e.nombre AS empresa, e.pais
FROM ofertas o
JOIN empresas e ON e.id = o.empresa_id
WHERE o.seniority = 'senior'
ORDER BY e.nombre, o.titulo;
```

RESULTADO EN MYSQL 8.0.46

titulo	empresa	pais
Data Engineer	Cumbre Data	MX
Frontend Senior	Cumbre Data	MX
Senior Frontend Engineer	Delta Pay	BR
Tech Lead Frontend	Mate Cloud	AR
Frontend React Senior	Nubelia	AR
Frontend Engineer	Pagos Sur	UY
Frontend React + TypeScript	Yerba Labs	PY

7 filas

- `JOIN` (o `INNER JOIN`, es lo mismo) combina cada oferta con **la empresa cuyo `id` coincide** según la condición del `ON`.

- `o` y `e` son **alias de tabla**: evitan escribir `ofertas.titulo` y desambiguan columnas con el mismo nombre (las dos tablas tienen `id`).
- Una fila que **no tiene pareja** en la otra tabla **desaparece** del resultado. Acá no pasa porque toda oferta tiene empresa, pero mirá qué pasa al revés.

LEFT JOIN: quedarse con todas las filas de la izquierda

¿Cuántas ofertas tiene cada empresa? Con `JOIN`, Quebracho AI (que no publicó ninguna) no aparece:

SQL

```sql
SELECT e.nombre, COUNT(*) AS ofertas
FROM empresas e
JOIN ofertas o ON o.empresa_id = e.id
GROUP BY e.id, e.nombre
ORDER BY ofertas DESC, e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	ofertas
Cumbre Data	2
Delta Pay	2
Mate Cloud	2
Nubelia	2
Pagos Sur	2
Remoto.dev	2
Andes Labs	1
Tereré Tech	1
Yerba Labs	1

9 filas

Con `LEFT JOIN` se conservan **todas** las filas de la tabla de la izquierda. Si no hay pareja, las columnas de la derecha vienen en `NULL`:

SQL

```sql
SELECT e.nombre, o.titulo
FROM empresas e
LEFT JOIN ofertas o ON o.empresa_id = e.id
WHERE e.pais = 'PY'
ORDER BY e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	titulo
Quebracho AI	*NULL*
Tereré Tech	Fullstack Next.js
Yerba Labs	Frontend React + TypeScript

3 filas

⚠️ COUNT(*) vs COUNT(columna) con LEFT JOIN

Quebracho AI tiene **una** fila (con `titulo` en `NULL`), así que `COUNT(*)` da 1. Para contar ofertas reales, contá una columna de la derecha: `COUNT(o.id)` ignora los `NULL` y da 0.

SQL

```sql
SELECT e.nombre,
       COUNT(*) AS mal,
       COUNT(o.id) AS bien
FROM empresas e
LEFT JOIN ofertas o ON o.empresa_id = e.id
WHERE e.pais = 'PY'
GROUP BY e.id, e.nombre
ORDER BY e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	mal	bien
Quebracho AI	1	0
Tereré Tech	1	1
Yerba Labs	1	1

3 filas

Buscar lo que falta

Un patrón clásico de entrevista: **las filas que no tienen pareja**. `LEFT JOIN` + `WHERE derecha.id IS NULL` (un *anti-join*):

SQL

```sql
-- Ofertas remotas a las que todavía no te postulaste
SELECT o.id, o.titulo
FROM ofertas o
LEFT JOIN postulaciones p ON p.oferta_id = o.id
WHERE p.id IS NULL
  AND o.modalidad = 'remoto'
ORDER BY o.id;
```

RESULTADO EN MYSQL 8.0.46

id	titulo
2	Fullstack Node
13	Backend Node.js
15	Junior Frontend

3 filas

SQL

```sql
-- Tecnologías que ninguna oferta pide
SELECT t.nombre
FROM tecnologias t
LEFT JOIN oferta_tecnologia ot ON ot.tecnologia_id = t.id
WHERE ot.oferta_id IS NULL;
```

RESULTADO EN MYSQL 8.0.46

nombre
Vue

1 fila

Lo mismo se puede escribir con `NOT EXISTS`, que se ve en subconsultas.

¿La condición va en el ON o en el WHERE?

Con `INNER JOIN` da igual. Con `LEFT JOIN`, **no**. Queremos todas las empresas de Paraguay y México con su cantidad de ofertas **remotas**, aunque sea 0:

× En el WHERE: se pierden empresas

SQL

```sql
SELECT e.nombre, COUNT(o.id) AS remotas
FROM empresas e
LEFT JOIN ofertas o ON o.empresa_id = e.id
WHERE e.pais IN ('PY', 'MX')
  AND o.modalidad = 'remoto'
GROUP BY e.id, e.nombre
ORDER BY remotas DESC, e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	remotas
Tereré Tech	1
Yerba Labs	1

2 filas

✓ En el ON: se conservan todas

SQL

```sql
SELECT e.nombre, COUNT(o.id) AS remotas
FROM empresas e
LEFT JOIN ofertas o ON o.empresa_id = e.id
                   AND o.modalidad = 'remoto'
WHERE e.pais IN ('PY', 'MX')
GROUP BY e.id, e.nombre
ORDER BY remotas DESC, e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	remotas
Tereré Tech	1
Yerba Labs	1
Cumbre Data	0
Quebracho AI	0

4 filas

En el `WHERE`, la condición se evalúa **después** del join: las dos ofertas de Cumbre Data son híbrida y presencial, así que se descartan; para Quebracho AI `o.modalidad` es `NULL`, y la fila también se descarta. El `LEFT JOIN` terminó funcionando como un `INNER JOIN`. En el `ON`, la condición decide **qué filas de la derecha se emparejan**: si ninguna cumple, la empresa queda igual, con `NULL` a la derecha y 0 en el conteo.

Regla: en un `LEFT JOIN`, las condiciones sobre la tabla de la derecha van en el `ON`; las de la izquierda, en el `WHERE`.

Muchos a muchos

Una oferta pide varias tecnologías y una tecnología aparece en varias ofertas. Eso se modela con una **tabla intermedia** (`oferta_tecnologia`), y para leerla hacen falta dos joins:

SQL

```sql
SELECT o.titulo, t.nombre AS tecnologia
FROM ofertas o
JOIN oferta_tecnologia ot ON ot.oferta_id = o.id
JOIN tecnologias t ON t.id = ot.tecnologia_id
WHERE o.empresa_id = 1
ORDER BY o.titulo, t.nombre;
```

RESULTADO EN MYSQL 8.0.46

titulo	tecnologia
Frontend React Senior	React
Frontend React Senior	Redux
Frontend React Senior	TypeScript
Fullstack Node	MySQL
Fullstack Node	Node.js
Fullstack Node	TypeScript

6 filas

Con `GROUP_CONCAT`, una fila por oferta:

SQL

```sql
SELECT o.titulo,
       GROUP_CONCAT(t.nombre ORDER BY t.nombre SEPARATOR ', ') AS stack
FROM ofertas o
JOIN oferta_tecnologia ot ON ot.oferta_id = o.id
JOIN tecnologias t ON t.id = ot.tecnologia_id
WHERE o.seniority = 'senior'
GROUP BY o.id, o.titulo
ORDER BY o.id;
```

RESULTADO EN MYSQL 8.0.46

titulo	stack
Frontend React Senior	React, Redux, TypeScript
Frontend Engineer	Next.js, React, TypeScript
Tech Lead Frontend	AWS, Next.js, React, TypeScript
Frontend React + TypeScript	React, TypeScript
Data Engineer	AWS, PostgreSQL, Python
Frontend Senior	React, Redux
Senior Frontend Engineer	Next.js, React, TypeScript

7 filas

Otra pregunta típica: **las ofertas que piden React *y* TypeScript**. Un `WHERE t.nombre = 'React' AND t.nombre = 'TypeScript'` no devuelve nada (una fila no puede tener dos nombres). Se filtran las filas con cualquiera de las dos y se pide que el grupo tenga las dos:

SQL

```sql
SELECT o.id, o.titulo
FROM ofertas o
JOIN oferta_tecnologia ot ON ot.oferta_id = o.id
JOIN tecnologias t ON t.id = ot.tecnologia_id
WHERE t.nombre IN ('React', 'TypeScript')
GROUP BY o.id, o.titulo
HAVING COUNT(*) = 2
ORDER BY o.id;
```

RESULTADO EN MYSQL 8.0.46

id	titulo
1	Frontend React Senior
3	Frontend Engineer

id	titulo
4	React Native Developer
7	Tech Lead Frontend
9	Frontend React + TypeScript
12	Senior Frontend Engineer
14	React Developer (LatAm)

7 filas

Varios joins seguidos

Se encadenan tantos como hagan falta. Tus postulaciones, con la oferta, la empresa y cuántas entrevistas tuviste:

SQL

```sql
SELECT e.nombre AS empresa,
       o.titulo,
       p.estado,
       COUNT(en.id) AS entrevistas
FROM postulaciones p
JOIN ofertas o ON o.id = p.oferta_id
JOIN empresas e ON e.id = o.empresa_id
LEFT JOIN entrevistas en ON en.postulacion_id = p.id
GROUP BY p.id, e.nombre, o.titulo, p.estado
ORDER BY entrevistas DESC, empresa;
```

RESULTADO EN MYSQL 8.0.46

empresa	titulo	estado	entrevistas
Nubelia	Frontend React Senior	entrevista	3
Pagos Sur	Frontend Engineer	oferta	3
Tereré Tech	Fullstack Next.js	entrevista	2
Yerba Labs	Frontend React + TypeScript	entrevista	1
Andes Labs	Desarrollador React	rechazado	0
Delta Pay	Senior Frontend Engineer	postulado	0
Mate Cloud	Tech Lead Frontend	postulado	0
Remoto.dev	React Developer (LatAm)	rechazado	0

8 filas

Fijate en el `LEFT JOIN entrevistas`: con `JOIN`, las postulaciones sin entrevistas no aparecerían.

Otros joins

Self join. Una tabla unida consigo misma, con dos alias. Pares de ofertas de la misma empresa:

SQL

```sql
SELECT a.titulo AS oferta, b.titulo AS otra_de_la_misma_empresa
FROM ofertas a
JOIN ofertas b ON b.empresa_id = a.empresa_id
              AND b.id > a.id
ORDER BY a.id, b.id;
```

RESULTADO EN MYSQL 8.0.46

oferta	otra_de_la_misma_empresa
Frontend React Senior	Fullstack Node
Frontend Engineer	React Native Developer
Frontend Junior	Tech Lead Frontend
Data Engineer	Frontend Senior
Senior Frontend Engineer	Backend Node.js
React Developer (LatAm)	Junior Frontend

6 filas

El `b.id > a.id` evita emparejar una oferta consigo misma y que cada par aparezca dos veces. El ejemplo clásico de self join es una tabla `empleados` con `jefe_id` que apunta a la misma tabla.

RIGHT JOIN es un `LEFT JOIN` con las tablas al revés. Casi nadie lo usa: se da vuelta el orden y se escribe `LEFT JOIN`.

CROSS JOIN combina todas las filas con todas (10 empresas × 10 tecnologías = 100 filas). Sirve para generar combinaciones, rara vez más.

FULL OUTER JOIN (todas las filas de los dos lados) **no existe en MySQL**. Se emula con un `UNION` de un `LEFT JOIN` y un `RIGHT JOIN`.

UNION: apilar resultados

`UNION` pone el resultado de una consulta **debajo** del de otra. Tienen que tener la misma cantidad de columnas:

SQL

```sql
SELECT 'empresa' AS tipo, nombre FROM empresas WHERE pais = 'UY'
UNION ALL
SELECT 'tecnologia', nombre FROM tecnologias WHERE nombre LIKE 'R%';
```

RESULTADO EN MYSQL 8.0.46

tipo	nombre
empresa	Pagos Sur
tecnologia	React
tecnologia	Redux

3 filas

`UNION` elimina filas duplicadas (y para eso tiene que ordenarlas o compararlas); `UNION ALL` las deja. Si sabés que no hay duplicados o no te importan, `UNION ALL` es más rápido.

> 🎤 **En una entrevista**
>
> "¿Qué diferencia hay entre INNER JOIN y LEFT JOIN?" → "INNER JOIN devuelve solo las filas que tienen pareja en las dos tablas. LEFT JOIN devuelve todas las de la izquierda y, si no hay pareja, completa las columnas de la derecha con NULL. Con eso se buscan filas sin relación: LEFT JOIN y WHERE derecha.id IS NULL. Y en un LEFT JOIN, filtrar la tabla de la derecha en el WHERE lo convierte en un INNER JOIN: esa condición va en el ON."

Parte 2 · Intermedio

Consultas de varios pasos, performance, concurrencia y diseño: lo que separa a un junior de un semi senior.

Capítulo 8

Subconsultas y CTE

Subconsultas en el WHERE, el FROM y el SELECT, EXISTS, WITH y CTE recursivas.

Una **subconsulta** es un `SELECT` adentro de otro. Una **CTE** (`WITH`) es lo mismo pero con nombre, escrita arriba, y se lee mucho mejor. Con las dos se resuelven en una sola consulta cosas que si no harían falta dos.

Subconsulta que devuelve un valor

Las ofertas que pagan más que el promedio:

SQL

```sql
SELECT titulo, salario_max
FROM ofertas
WHERE salario_max > (SELECT AVG(salario_max) FROM ofertas)
ORDER BY salario_max DESC;
```

RESULTADO EN MYSQL 8.0.46

titulo	salario_max
Senior Frontend Engineer	6000
Tech Lead Frontend	5500
Frontend Engineer	4500
Data Engineer	4200
Frontend React Senior	4000

5 filas

La subconsulta entre paréntesis se ejecuta y su resultado (un número) se usa en la comparación. Si devolviera más de una fila, sería un error:

SQL

```sql
SELECT titulo
FROM ofertas
WHERE empresa_id = (SELECT id FROM empresas WHERE pais = 'PY');
```

RESULTADO EN MYSQL 8.0.46

```
ERROR 1242 (21000): Subquery returns more than 1 row
```

IN: una lista de valores

Si la subconsulta devuelve una columna con varias filas, se usa con `IN`:

SQL

```sql
SELECT titulo, empresa_id
FROM ofertas
WHERE empresa_id IN (SELECT id FROM empresas WHERE pais = 'PY')
ORDER BY id;
```

RESULTADO EN MYSQL 8.0.46

titulo	empresa_id
Fullstack Next.js	5
Frontend React + TypeScript	6

2 filas

> ⚠️ **NOT IN y los NULL**
>
> `NOT IN` con una subconsulta que devuelve algún `NULL` **no devuelve ninguna fila**: `x NOT IN (1, NULL)` es `x <> 1 AND x <> NULL`, y lo segundo nunca es verdadero. Por eso, para "lo que no está", es más seguro `NOT EXISTS` o el `LEFT JOIN … IS NULL`.

SQL

```sql
SELECT COUNT(*) AS empresas_sin_ese_tamanio
FROM empresas
WHERE empleados NOT IN (SELECT empleados FROM empresas WHERE pais =
  'US');
```

RESULTADO EN MYSQL 8.0.46

empresas_sin_ese_tamanio
0

1 fila

> Remoto.dev (la única de US) tiene `empleados` en `NULL`, así que la lista es `(NULL)` y el resultado es 0, cuando uno esperaría las otras 9 empresas.

EXISTS y subconsultas correlacionadas

Una subconsulta **correlacionada** usa columnas de la consulta de afuera: conceptualmente, se evalúa una vez por cada fila. `EXISTS` pregunta si devuelve **al menos una fila**:

SQL

```sql
-- Empresas con al menos una oferta remota senior
SELECT e.nombre
FROM empresas e
WHERE EXISTS (
  SELECT 1
  FROM ofertas o
  WHERE o.empresa_id = e.id
    AND o.modalidad = 'remoto'
```

```
    AND o.seniority = 'senior'
)
ORDER BY e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre
Delta Pay
Mate Cloud
Nubelia
Pagos Sur
Yerba Labs

5 filas

SQL

```
-- Ofertas a las que no te postulaste (igual que el LEFT JOIN ... IS
  NULL)
SELECT o.id, o.titulo
FROM ofertas o
WHERE NOT EXISTS (SELECT 1 FROM postulaciones p WHERE p.oferta_id = o.id)
ORDER BY o.id;
```

RESULTADO EN MYSQL 8.0.46

id	titulo
2	Fullstack Node
4	React Native Developer
6	Frontend Junior
10	Data Engineer
11	Frontend Senior
13	Backend Node.js
15	Junior Frontend

7 filas

El `SELECT 1` es una convención: a `EXISTS` no le importan las columnas, solo si hay filas.

Una correlacionada también puede ir en el `SELECT`, para calcular un valor por fila:

SQL

```sql
SELECT e.nombre,
       (SELECT COUNT(*) FROM ofertas o WHERE o.empresa_id = e.id) AS
         ofertas,
       (SELECT MAX(o.salario_max) FROM ofertas o WHERE o.empresa_id =
         e.id) AS mejor_salario
FROM empresas e
WHERE e.pais IN ('AR', 'PY')
ORDER BY e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	ofertas	mejor_salario
Mate Cloud	2	5500
Nubelia	2	4000
Quebracho AI	0	*NULL*
Tereré Tech	1	2800
Yerba Labs	1	3300

5 filas

Funciona, pero con muchas filas suele ser más claro y rápido un `LEFT JOIN` con `GROUP BY`.

Subconsulta en el FROM (tabla derivada)

El resultado de un `SELECT` se puede usar como si fuera una tabla. Necesita un alias:

SQL

```sql
SELECT seniority, ROUND(AVG(ofertas_por_empresa), 1) AS promedio
FROM (
  SELECT empresa_id, seniority, COUNT(*) AS ofertas_por_empresa
  FROM ofertas
  GROUP BY empresa_id, seniority
) AS conteo
GROUP BY seniority
ORDER BY seniority;
```

RESULTADO EN MYSQL 8.0.46

seniority	promedio
junior	1.0
semi	1.0
senior	1.2

3 filas

CTE: WITH

Lo mismo, con nombre y arriba. Se lee de arriba hacia abajo, como pasos:

SQL

```sql
WITH por_empresa AS (
  SELECT empresa_id,
         COUNT(*) AS ofertas,
         MAX(salario_max) AS mejor
  FROM ofertas
  WHERE modalidad = 'remoto'
  GROUP BY empresa_id
)
SELECT e.nombre, pe.ofertas, pe.mejor
FROM por_empresa pe
JOIN empresas e ON e.id = pe.empresa_id
WHERE pe.mejor >= 3500
ORDER BY pe.mejor DESC;
```

RESULTADO EN MYSQL 8.0.46

nombre	ofertas	mejor
Delta Pay	2	6000
Mate Cloud	1	5500
Pagos Sur	1	4500
Nubelia	2	4000
Remoto.dev	2	3500

5 filas

Se pueden definir varias, separadas por coma, y cada una puede usar las anteriores:

SQL

```sql
WITH remotas AS (
  SELECT * FROM ofertas WHERE modalidad = 'remoto'
),
promedio AS (
  SELECT AVG(salario_max) AS valor FROM remotas
)
SELECT r.titulo, r.salario_max, ROUND(p.valor) AS promedio_remotas
FROM remotas r
CROSS JOIN promedio p
WHERE r.salario_max > p.valor
ORDER BY r.salario_max DESC;
```

RESULTADO EN MYSQL 8.0.46

titulo	salario_max	promedio_remotas
Senior Frontend Engineer	6000	3844
Tech Lead Frontend	5500	3844
Frontend Engineer	4500	3844
Frontend React Senior	4000	3844

4 filas

> 💡 **Cuándo usar CTE**
>
> Siempre que una consulta tenga "pasos" (primero calculo esto, después lo uso para aquello). Es la forma más legible de escribir SQL complejo, y en una entrevista de live coding ayuda a explicar lo que vas haciendo. MySQL las tiene desde la versión 8.0.

CTE recursiva

Una CTE `RECURSIVE` se referencia a sí misma: arranca con una fila y va agregando filas hasta que la condición corta. Sirve para recorrer jerarquías (categorías, comentarios con respuestas, organigramas) o para **generar series**:

SQL

```sql
WITH RECURSIVE semanas AS (
  SELECT DATE('2026-08-31') AS desde
```

```
  UNION ALL
  SELECT desde + INTERVAL 7 DAY
  FROM semanas
  WHERE desde < '2026-10-19'
)
SELECT s.desde, COUNT(p.id) AS postulaciones
FROM semanas s
LEFT JOIN postulaciones p
       ON p.fecha >= s.desde AND p.fecha < s.desde + INTERVAL 7 DAY
GROUP BY s.desde
ORDER BY s.desde;
```

RESULTADO EN MYSQL 8.0.46

desde	postulaciones
2026-08-31	1
2026-09-07	0
2026-09-14	1
2026-09-21	0
2026-09-28	2
2026-10-05	1
2026-10-12	1
2026-10-19	2

8 filas

Las semanas sin postulaciones aparecen con 0 gracias al `LEFT JOIN` contra la serie. Sin la serie, esas semanas no estarían en el resultado: es un truco muy usado para reportes y gráficos.

> **🎤 En una entrevista**
>
> "¿Subconsulta, JOIN o CTE?" → "Para traer columnas de otra tabla, JOIN. Para filtrar por existencia, EXISTS o NOT EXISTS (cuidado con NOT IN y los NULL). Para consultas con varios pasos, CTE, que es una subconsulta con nombre y se lee mejor. El optimizador de MySQL 8 muchas veces reescribe una forma en otra, así que primero escribo lo más claro y después miro el EXPLAIN si es lento."

Capítulo 9

Funciones de ventana

OVER y PARTITION BY, rankings, el mejor de cada grupo, acumulados, LAG y LEAD.

Las **funciones de ventana** calculan algo sobre un grupo de filas, como `GROUP BY`, pero **sin juntar las filas**: cada fila se queda en el resultado y gana una columna con el cálculo. Son la herramienta para rankings, "el mejor de cada grupo", acumulados y comparar una fila con la anterior. MySQL las tiene desde la 8.0.

OVER: agregar sin agrupar

SQL

```sql
SELECT titulo,
       seniority,
       salario_max,
       ROUND(AVG(salario_max) OVER (PARTITION BY seniority)) AS
         promedio_seniority,
       MAX(salario_max) OVER () AS maximo_total
FROM ofertas
WHERE salario_max IS NOT NULL
ORDER BY seniority, salario_max DESC;
```

RESULTADO EN MYSQL 8.0.46

titulo	seniority	salario_max	promedio_seniority	maximo_total
Junior Frontend	junior	2000	1800	6000
Frontend Junior	junior	1600	1800	6000
React Developer (LatAm)	semi	3500	3125	6000
React Native Developer	semi	3200	3125	6000
Fullstack Node	semi	3000	3125	6000
Fullstack Next.js	semi	2800	3125	6000
Senior Frontend Engineer	senior	6000	4414	6000
Tech Lead Frontend	senior	5500	4414	6000

titulo	seniority	salario_max	promedio_seniority	maximo_total
Frontend Engineer	senior	4500	4414	6000
Data Engineer	senior	4200	4414	6000
Frontend React Senior	senior	4000	4414	6000
Frontend Senior	senior	3400	4414	6000
Frontend React + TypeScript	senior	3300	4414	6000

13 filas

- `OVER (...)` convierte una agregación (`AVG`, `MAX`, `SUM`, `COUNT`...) en función de ventana.
- `PARTITION BY seniority` define la **ventana**: las filas con el mismo seniority. Es el equivalente al `GROUP BY`, pero cada fila sigue estando.
- `OVER ()` vacío: la ventana es todo el resultado.

Con eso se compara cada fila contra su grupo sin subconsultas: "cuánto se aleja esta oferta del promedio de su seniority".

Rankings: ROW_NUMBER, RANK y DENSE_RANK

Las tres numeran filas según un `ORDER BY` dentro de la ventana. La diferencia está en los **empates** (hay dos ofertas con salario mínimo 3000 y dos con 2500):

SQL

```sql
SELECT titulo,
       salario_min,
       ROW_NUMBER() OVER (ORDER BY salario_min DESC) AS row_num,
       RANK()       OVER (ORDER BY salario_min DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salario_min DESC) AS dense
FROM ofertas
WHERE salario_min >= 2500
ORDER BY salario_min DESC, row_num;
```

RESULTADO EN MYSQL 8.0.46

titulo	salario_min	row_num	rnk	dense
Tech Lead Frontend	4500	1	1	1
Senior Frontend Engineer	4000	2	2	2

titulo	salario_min	row_num	rnk	dense
Frontend Engineer	3500	3	3	3
Frontend React Senior	3000	4	4	4
Data Engineer	3000	5	4	4
Frontend React + TypeScript	2800	6	6	5
Frontend Senior	2600	7	7	6
React Native Developer	2500	8	8	7
React Developer (LatAm)	2500	9	8	7

9 filas

Función	Con empate	Ejemplo
`ROW_NUMBER()`	Números distintos (el orden entre empatados es arbitrario)	1, 2, 3, 4, 5
`RANK()`	Mismo número, y **salta** los siguientes	1, 2, 3, 3, 5
`DENSE_RANK()`	Mismo número, **sin saltos**	1, 2, 3, 3, 4

El mejor de cada grupo

La pregunta de entrevista más común con ventanas: **la oferta mejor paga de cada seniority** (o "el último pedido de cada cliente", "el empleado que más gana por departamento"). Las funciones de ventana no se pueden usar en el `WHERE`:

SQL

```sql
SELECT titulo, seniority, salario_max
FROM ofertas
WHERE ROW_NUMBER() OVER (PARTITION BY seniority ORDER BY salario_max
  DESC) = 1;
```

RESULTADO EN MYSQL 8.0.46

```
ERROR 3593 (HY000): You cannot use the window function 'row_number' in
this context.'
```

Porque el `WHERE` se evalúa **antes** que las ventanas. La solución: calcular el número en una CTE y filtrar afuera.

SQL

```sql
WITH ranking AS (
  SELECT titulo,
         seniority,
         salario_max,
         ROW_NUMBER() OVER (PARTITION BY seniority ORDER BY salario_max
           DESC) AS puesto
  FROM ofertas
  WHERE salario_max IS NOT NULL
)
SELECT seniority, titulo, salario_max
FROM ranking
WHERE puesto = 1
ORDER BY salario_max DESC;
```

RESULTADO EN MYSQL 8.0.46

seniority	titulo	salario_max
senior	Senior Frontend Engineer	6000
semi	React Developer (LatAm)	3500
junior	Junior Frontend	2000

3 filas

Cambiando `puesto = 1` por `puesto <= 2` tenés el **top 2 de cada grupo**. Si querés que los empates entren todos, usá `RANK()` en lugar de `ROW_NUMBER()`.

Acumulados

Con `ORDER BY` adentro del `OVER`, `SUM` y `COUNT` se vuelven **acumulados**: suman desde la primera fila hasta la actual.

SQL

```sql
SELECT fecha,
       estado,
       COUNT(*) OVER (ORDER BY fecha) AS postulaciones_hasta_hoy
FROM postulaciones
ORDER BY fecha;
```

RESULTADO EN MYSQL 8.0.46

fecha	estado	postulaciones_hasta_hoy
2026-09-05	entrevista	1
2026-09-16	rechazado	2
2026-10-02	oferta	3
2026-10-04	entrevista	4
2026-10-10	rechazado	5
2026-10-13	postulado	6
2026-10-19	entrevista	7
2026-10-21	postulado	8

8 filas

Y con `PARTITION BY`, el acumulado se reinicia en cada grupo.

> **ℹ️ El marco de la ventana**
>
> Cuando hay `ORDER BY`, la ventana por defecto va "desde el principio hasta la fila actual". Se puede cambiar con un **marco**: por ejemplo, un promedio móvil de las últimas 3 filas es `AVG(x) OVER (ORDER BY fecha ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`.

LAG y LEAD: la fila anterior y la siguiente

`LAG(columna)` trae el valor de la fila **anterior** de la ventana; `LEAD(columna)`, el de la **siguiente**. Cuántos días pasaron entre cada entrevista de un mismo proceso:

SQL

```sql
SELECT postulacion_id,
       tipo,
       DATE(fecha) AS dia,
       DATEDIFF(fecha, LAG(fecha) OVER (PARTITION BY postulacion_id ORDER
         BY fecha)) AS dias_desde_la_anterior,
       LEAD(tipo) OVER (PARTITION BY postulacion_id ORDER BY fecha) AS
         siguiente
FROM entrevistas
ORDER BY postulacion_id, fecha;
```

RESULTADO EN MYSQL 8.0.46

postulacion_id	tipo	dia	dias_desde_la_anterior	siguiente
1	rrhh	2026-09-10	*NULL*	tecnica
1	tecnica	2026-09-17	7	live coding
1	live coding	2026-09-24	7	*NULL*
2	rrhh	2026-10-06	*NULL*	tecnica
2	tecnica	2026-10-09	3	final
2	final	2026-10-14	5	*NULL*
5	rrhh	2026-10-23	*NULL*	*NULL*
8	rrhh	2026-10-07	*NULL*	tecnica
8	tecnica	2026-10-15	8	*NULL*

9 filas

La primera entrevista de cada postulación no tiene anterior, y la última no tiene siguiente: dan `NULL`.

Una ventana con nombre

Si varias columnas usan la misma ventana, se define una vez con `WINDOW`:

SQL

```sql
SELECT titulo,
       salario_max,
       RANK() OVER w AS puesto,
       salario_max - LAG(salario_max) OVER w AS diferencia
FROM ofertas
WHERE seniority = 'senior'
WINDOW w AS (ORDER BY salario_max DESC)
ORDER BY puesto;
```

RESULTADO EN MYSQL 8.0.46

titulo	salario_max	puesto	diferencia
Senior Frontend Engineer	6000	1	*NULL*
Tech Lead Frontend	5500	2	-500
Frontend Engineer	4500	3	-1000
Data Engineer	4200	4	-300
Frontend React Senior	4000	5	-200
Frontend Senior	3400	6	-600
Frontend React + TypeScript	3300	7	-100

7 filas

🎤 En una entrevista

"Traeme el segundo salario más alto." → Con ventanas: `DENSE_RANK() OVER (ORDER BY salario DESC)` en una CTE y filtrar `= 2` (con `DENSE_RANK` los empates en el primero no corren el segundo). Sin ventanas: `SELECT MAX(salario) FROM t WHERE salario < (SELECT MAX(salario) FROM t)`. Y si piden "el N-ésimo", la de ventanas es la que escala.

Capítulo 10

Índices y EXPLAIN

Cómo funciona un índice, cómo leer un EXPLAIN, índices compuestos y qué los rompe.

Con 15 filas todo es instantáneo. Con 5 millones, la misma consulta puede tardar segundos o milisegundos según haya o no un **índice**. Es el tema de performance que más preguntan en entrevistas de backend.

Qué es un índice

Sin índice, para encontrar las ofertas publicadas después de cierta fecha, MySQL tiene que **leer todas las filas** y quedarse con las que cumplen (un *full table scan*). Un índice es una estructura aparte (un **árbol B+**) que guarda los valores de una columna **ordenados**, cada uno con la ubicación de su fila. Como el índice de un libro: en lugar de leer el libro entero, buscás la palabra en orden alfabético y vas a la página.

- Buscar un valor en un árbol con un millón de valores toma unos pocos saltos, no un millón de lecturas.
- Como está ordenado, también sirve para **rangos** (`>`, `BETWEEN`, `LIKE 'abc%'`) y para **ordenar** sin un paso extra.

> **ℹ️ La clave primaria es especial**
>
> En InnoDB (el motor por defecto de MySQL), la tabla misma está guardada **ordenada por la clave primaria** (un *clustered index*). Los demás índices guardan el valor indexado más la clave primaria, y con ella van a buscar la fila. Por eso una PK chica (un `INT` o `BIGINT` autoincremental) hace más livianos a todos los índices.

EXPLAIN: cómo piensa ejecutar MySQL

`EXPLAIN` antes de un `SELECT` no lo ejecuta: muestra el **plan**.

SQL

```sql
EXPLAIN SELECT titulo FROM ofertas WHERE publicada >= '2026-10-20';
```

RESULTADO EN MYSQL 8.0.46

```
*** fila 1 ***
id            1
select_type   SIMPLE
table         ofertas
partitions    NULL
type          ALL
possible_keys NULL
key           NULL
key_len       NULL
ref           NULL
rows          15
filtered      33.33
Extra         Using where
```

Las columnas que importan:

Columna	Qué dice
`type`	Cómo accede a la tabla. De peor a mejor: `ALL` (lee todo), `index` (lee todo el índice), `range` (un rango del índice), `ref` (busca un valor), `eq_ref` (una fila por join), `const` (una fila por PK o UNIQUE)
`key`	El índice que eligió (`NULL` : ninguno)
`rows`	Cuántas filas estima que va a leer
`Extra`	Detalles: `Using where` (filtra después de leer), `Using index` (no necesita ir a la tabla), `Using filesort` (ordena aparte), `Using temporary` (tabla temporal)

Acá: `type: ALL` y `key: NULL`. Lee las 15 filas para devolver 3.

Crear un índice

SQL

```sql
CREATE INDEX idx_ofertas_publicada ON ofertas (publicada);

EXPLAIN SELECT titulo FROM ofertas WHERE publicada >= '2026-10-20';
```

RESULTADO EN MYSQL 8.0.46

Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0

*** fila 1 ***	
id	1
select_type	SIMPLE
table	ofertas
partitions	*NULL*
type	range
possible_keys	idx_ofertas_publicada
key	idx_ofertas_publicada
key_len	3
ref	*NULL*
rows	3
filtered	100
Extra	Using index condition

Ahora `type: range`, `key: idx_ofertas_publicada` y `rows: 3`: va directo al rango del índice. Con una tabla grande, esa es la diferencia entre segundos y milisegundos.

Para ver los índices de una tabla: `SHOW INDEX FROM ofertas`, o más resumido:

SQL

```sql
SELECT index_name, column_name, seq_in_index, non_unique
FROM information_schema.statistics
WHERE table_schema = DATABASE()
  AND table_name = 'ofertas'
ORDER BY index_name, seq_in_index;
```

RESULTADO EN MYSQL 8.0.46

INDEX_NAME	COLUMN_NAME	SEQ_IN_INDEX	NON_UNIQUE
empresa_id	empresa_id	1	1
PRIMARY	id	1	0

2 filas

Hay dos que no creaste: `PRIMARY` (la clave primaria siempre tiene índice) y `empresa_id`, que InnoDB crea solo para la **clave foránea**. Las restricciones `UNIQUE` también crean un índice.

Índices compuestos: el orden importa

Un índice puede tener varias columnas. Se ordena por la primera, y para valores iguales de la primera, por la segunda (como una guía telefónica: apellido y después nombre).

SQL

```sql
CREATE INDEX idx_modalidad_salario ON ofertas (modalidad, salario_max);

EXPLAIN SELECT titulo FROM ofertas
WHERE modalidad = 'remoto' AND salario_max > 4000;

EXPLAIN SELECT titulo FROM ofertas
WHERE salario_max > 4000;
```

RESULTADO EN MYSQL 8.0.46

Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0

*** fila 1 ***	
id	1
select_type	SIMPLE
table	ofertas
partitions	*NULL*
type	range
possible_keys	idx_modalidad_salario
key	idx_modalidad_salario
key_len	6
ref	*NULL*
rows	3
filtered	100

Extra	Using index condition

*** fila 1 ***	
id	1
select_type	SIMPLE
table	ofertas
partitions	*NULL*
type	ALL
possible_keys	*NULL*
key	*NULL*
key_len	*NULL*
ref	*NULL*
rows	15
filtered	33.33
Extra	Using where

La primera usa el índice. La segunda **no**: sin saber la modalidad, los salarios están desparramados por todo el índice, igual que en la guía telefónica no podés buscar a todos los que se llaman "Juan" sin leerla entera. Es la **regla del prefijo izquierdo**: un índice `(a, b, c)` sirve para filtrar por `a`, por `a, b` o por `a, b, c`, no por `b` ni por `c` solos.

> 💡 **Cómo ordenar las columnas**
>
> Primero las que se comparan por **igualdad** (`=`, `IN`), después la del **rango** o la del **ORDER BY**. Un índice `(empresa_id, publicada)` resuelve `WHERE empresa_id = 8 ORDER BY publicada DESC` sin ordenar nada: lee el pedazo de la empresa 8, que ya está ordenado por fecha.

Cómo se rompe un índice

El índice guarda los valores de la columna **tal cual**. Si la consulta aplica una función sobre la columna, MySQL ya no puede buscar en el árbol:

× Función sobre la columna: ALL

SQL

```sql
CREATE INDEX idx_ofertas_publicada ON ofertas (publicada);
```

```sql
EXPLAIN SELECT titulo FROM ofertas
WHERE YEAR(publicada) = 2026 AND MONTH(publicada) = 10;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0
```

*** fila 1 ***	
id	1
select_type	SIMPLE
table	ofertas
partitions	*NULL*
type	ALL
possible_keys	*NULL*
key	*NULL*
key_len	*NULL*
ref	*NULL*
rows	15
filtered	100
Extra	Using where

✓ Rango sobre la columna: range

SQL

```sql
CREATE INDEX idx_ofertas_publicada ON ofertas (publicada);

EXPLAIN SELECT titulo FROM ofertas
WHERE publicada >= '2026-10-01' AND publicada < '2026-11-01';
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0
```

*** fila 1 ***	
id	1
select_type	SIMPLE
table	ofertas
partitions	*NULL*
type	range
possible_keys	idx_ofertas_publicada
key	idx_ofertas_publicada
key_len	3
ref	*NULL*
rows	9
filtered	100
Extra	Using index condition

Lo mismo pasa con:

- `LIKE '%React%'` : un comodín al **principio** obliga a revisar todo. `LIKE 'React%'` sí usa el índice (es un rango). Para buscar palabras dentro de textos existe el índice `FULLTEXT`.
- Operaciones: `WHERE salario_max * 12 > 50000` no usa el índice; `WHERE salario_max > 50000 / 12` sí.
- Comparar un texto con un número (`WHERE titulo = 15`): MySQL convierte cada fila a número y no puede buscar.

Índice que cubre la consulta

Si todas las columnas que pide la consulta están **en el índice**, MySQL no necesita ir a la tabla: responde leyendo solo el índice. `EXPLAIN` lo muestra como `Using index`.

SQL

```sql
CREATE INDEX idx_modalidad_salario ON ofertas (modalidad, salario_max);

EXPLAIN SELECT modalidad, salario_max FROM ofertas
WHERE modalidad = 'remoto';
```

RESULTADO EN MYSQL 8.0.46

Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0

*** fila 1 ***

id	1
select_type	SIMPLE
table	ofertas
partitions	*NULL*
type	ref
possible_keys	idx_modalidad_salario
key	idx_modalidad_salario
key_len	1
ref	const
rows	11
filtered	100
Extra	Using where; Using index

Los índices no son gratis

- Cada `INSERT`, `UPDATE` y `DELETE` tiene que actualizar **todos** los índices de la tabla. Muchos índices hacen más lentas las escrituras.
- Ocupan espacio (en disco y en la memoria del servidor).
- Una columna con pocos valores distintos (como `modalidad`, con 3) sola es un mal índice: cada valor coincide con un tercio de la tabla y leerla entera puede ser igual de rápido. Combinada con otra en un índice compuesto, sí puede servir.
- El optimizador decide. Con tablas chicas puede ignorar un índice porque leer todo es más barato.

Qué indexar

1. Las columnas de las **claves foráneas** (InnoDB ya lo hace) y las que usás en los `JOIN`.
2. Las columnas de los `WHERE` más frecuentes, y de los `ORDER BY` de listas paginadas.
3. Lo que muestre el **slow query log** (MySQL puede registrar las consultas que tardan más de N segundos). No indexes "por las dudas": medí.

`EXPLAIN ANALYZE` (MySQL 8.0.18+) **ejecuta** la consulta y muestra el plan con los tiempos y las filas reales de cada paso:

```
EXPLAIN ANALYZE SELECT titulo FROM ofertas WHERE publicada >= '2026-10-
  20';
-- -> Index range scan on ofertas using idx_ofertas_publicada over
  ('2026-10-20' <= publicada)
--    (cost=1.61 rows=3) (actual time=0.018..0.02 rows=3 loops=1)
```

🎤 En una entrevista

"Una consulta está lenta, ¿qué hacés?" → "Primero la reproduzco y miro el EXPLAIN (o EXPLAIN ANALYZE): si hay un type ALL sobre una tabla grande, rows muy alto, Using filesort o Using temporary. Después veo si falta un índice para el WHERE, el JOIN o el ORDER BY, o si la consulta rompe uno que existe (una función sobre la columna, un LIKE con % adelante). Si hace falta, creo un índice compuesto con las columnas de igualdad primero y la de rango o de orden al final, y vuelvo a medir. También reviso si trae columnas o filas de más: SELECT *, sin LIMIT, o un N+1 desde la aplicación."

Capítulo 11

Transacciones y concurrencia

COMMIT y ROLLBACK, ACID, niveles de aislamiento, bloqueos y deadlocks.

Una **transacción** agrupa varias operaciones para que se apliquen **todas o ninguna**. El ejemplo de manual es una transferencia: restar de una cuenta y sumar en otra. Si el servidor se cae entre las dos, no puede quedar la plata restada y no sumada.

START TRANSACTION, COMMIT y ROLLBACK

SQL

```sql
START TRANSACTION;

UPDATE postulaciones SET estado = 'oferta' WHERE id = 1;
UPDATE entrevistas SET resultado = 'paso'
WHERE postulacion_id = 1 AND resultado = 'pendiente';

COMMIT;

SELECT p.estado, COUNT(*) AS entrevistas, SUM(en.resultado = 'paso') AS
  pasadas
FROM postulaciones p
JOIN entrevistas en ON en.postulacion_id = p.id
WHERE p.id = 1
GROUP BY p.id, p.estado;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas
Query OK, 1 fila afectada · Rows matched: 1 Changed: 1 Warnings: 0
Query OK, 1 fila afectada · Rows matched: 1 Changed: 1 Warnings: 0
Query OK, 0 filas afectadas
```

estado	entrevistas	pasadas
oferta	3	3

1 fila

- `START TRANSACTION` (o `BEGIN`) abre la transacción.
- Los cambios se van haciendo, pero **nadie más los ve** hasta el `COMMIT`, que los confirma de una vez.
- `ROLLBACK` los deshace todos:

SQL

```sql
START TRANSACTION;

DELETE FROM entrevistas WHERE postulacion_id = 1;
SELECT COUNT(*) AS entrevistas FROM entrevistas;

ROLLBACK;

SELECT COUNT(*) AS entrevistas FROM entrevistas;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas
Query OK, 3 filas afectadas
```

entrevistas
6

1 fila

```
Query OK, 0 filas afectadas
```

entrevistas
9

1 fila

Adentro de la transacción ya se ven 6 entrevistas; después del `ROLLBACK` vuelven a ser 9.

> ℹ️ **autocommit**
>
> Fuera de una transacción explícita, MySQL trabaja en modo **autocommit**: cada sentencia es su propia transacción y se confirma sola al terminar. Por eso un `UPDATE` suelto se guarda sin escribir `COMMIT`.

> ⚠️ **Un error no deshace la transacción**
>
> Si una sentencia falla adentro de una transacción, MySQL descarta **esa** sentencia, pero las anteriores siguen pendientes y la transacción sigue abierta. Tu código tiene que hacer `ROLLBACK` (en Node, en el `catch`). Y las sentencias de estructura (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`) hacen un **COMMIT implícito**: no se pueden deshacer ni meter en una transacción.

`SAVEPOINT nombre` marca un punto intermedio y `ROLLBACK TO nombre` vuelve hasta ahí sin deshacer todo. Se usa poco.

ACID

Las cuatro garantías de una transacción, una pregunta clásica:

	Qué garantiza
Atomicidad	Se aplican todas las operaciones o ninguna.
Consistencia	La base pasa de un estado válido a otro: se respetan claves, `UNIQUE`, `CHECK` y claves foráneas.
Isolation (aislamiento)	Las transacciones que corren al mismo tiempo no se pisan (con matices: ver niveles abajo).
Durabilidad	Lo confirmado con `COMMIT` sobrevive a un corte de luz: ya está escrito en el log en disco.

Concurrencia: cuando dos transacciones tocan lo mismo

Con muchos usuarios a la vez, dos transacciones pueden leer y escribir las mismas filas. Los problemas posibles:

Problema	Qué pasa
Dirty read	Leés un cambio de otra transacción que **todavía no hizo commit** (y quizás nunca lo haga).
Non-repeatable read	Leés una fila dos veces en tu transacción y cambió en el medio.
Phantom read	Repetís una consulta y aparecen filas nuevas que insertó otra transacción.
Lost update	Dos transacciones leen el mismo valor, calculan y escriben: el último pisa al primero.

El **nivel de aislamiento** decide cuáles se evitan:

Nivel	Dirty	Non-repeatable	Phantom
`READ UNCOMMITTED`	puede pasar	puede pasar	puede pasar
`READ COMMITTED`	—	puede pasar	puede pasar
`REPEATABLE READ` (**default de MySQL**)	—	—	casi nunca (InnoDB lo evita en la mayoría de los casos)
`SERIALIZABLE`	—	—	—

SQL

```sql
SELECT @@transaction_isolation;
```

RESULTADO EN MYSQL 8.0.46

@@transaction_isolation
REPEATABLE-READ

1 fila

En `REPEATABLE READ`, cada transacción ve una **foto** de los datos tomada en su primera lectura: lo que otras confirmen después no le aparece. Es cómodo, pero no evita el *lost update*.

El lost update y cómo evitarlo

Dos personas reservan el último lugar de un taller (`cupos = 1`) al mismo tiempo:

```
-- Sesión A                                    -- Sesión B
START TRANSACTION;                             START TRANSACTION;
SELECT cupos FROM talleres WHERE id = 7;       SELECT cupos FROM talleres
  WHERE id = 7;
-- ve 1: hay lugar                             -- ve 1: hay lugar
UPDATE talleres SET cupos = 0 WHERE id = 7;
COMMIT;                                        UPDATE talleres SET cupos =
  0 WHERE id = 7;
                                               COMMIT;
-- Las dos reservaron el mismo lugar.
```

Tres soluciones, de la más simple a la más general:

1. Que la base haga la cuenta en un solo `UPDATE` atómico, con la condición adentro:

```
UPDATE talleres SET cupos = cupos - 1 WHERE id = 7 AND cupos > 0;
-- filas afectadas: 1 → reservaste; 0 → no quedaba lugar
```

2. Bloquear la fila al leerla con `SELECT … FOR UPDATE`. La segunda sesión se queda **esperando** en su `SELECT` hasta que la primera termine, y entonces lee el valor nuevo:

```
START TRANSACTION;
SELECT cupos FROM talleres WHERE id = 7 FOR UPDATE;   -- bloquea la fila
-- ... lógica de la app: si cupos > 0, reservar ...
UPDATE talleres SET cupos = cupos - 1 WHERE id = 7;
COMMIT;                                               -- libera el
  bloqueo
```

3. Bloqueo optimista con una columna `version`. No bloquea nada: al escribir, se verifica que nadie haya cambiado la fila desde que la leíste.

Es lo que usan muchos ORMs.

```
SELECT cupos, version FROM talleres WHERE id = 7;      -- 1, version 4
UPDATE talleres SET cupos = 0, version = version + 1
WHERE id = 7 AND version = 4;
-- 0 filas afectadas → alguien la cambió antes: volver a leer y
  reintentar
```

Deadlocks

Un **deadlock** pasa cuando la transacción A espera una fila que tiene bloqueada B, y B espera una que tiene A. Ninguna puede seguir. InnoDB lo detecta al instante, **cancela una** de las dos y le devuelve:

```
ERROR 1213 (40001): Deadlock found when trying to get lock; try
  restarting transaction
```

No es un bug de MySQL: es esperable con concurrencia. Para tenerlos menos, transacciones **cortas** y que bloqueen las filas **siempre en el mismo orden**; y la aplicación tiene que **reintentar** la transacción cuando recibe ese error.

> 🎤 **En una entrevista**
>
> "¿Qué es una transacción y para qué sirve?" → "Un grupo de operaciones que se aplican todas o ninguna: si algo falla en el medio, ROLLBACK y la base queda como estaba. Garantiza ACID. Por ejemplo, crear un pedido y descontar el stock: si falla lo segundo, no puede quedar el pedido creado. Para la concurrencia, MySQL usa REPEATABLE READ por defecto; cuando leo un valor para después escribirlo, uso un UPDATE atómico con la condición adentro, SELECT FOR UPDATE o bloqueo optimista con una columna version."

Capítulo 12

Diseño de tablas

Normalización, relaciones, ON DELETE, elegir tipos y decisiones típicas.

Escribir consultas es la mitad del trabajo. La otra mitad es **decidir qué tablas hay**: un mal diseño se paga en cada consulta, con datos duplicados que se contradicen. Este capítulo es lo que se pregunta en una entrevista cuando te dan un dominio ("diseñá la base de un e-commerce").

El problema: una planilla

Si la búsqueda laboral estuviera en una sola tabla, como en una planilla de cálculo:

oferta	empresa	pais_empresa	tecnologias	estado
Frontend React Senior	Nubelia	AR	React, TypeScript, Redux	entrevista
Fullstack Node	Nubelia	Argentina	Node.js, TypeScript, MySQL	
Frontend Engineer	Pagos Sur	UY	React, TypeScript, Next.js	oferta

Aparecen los problemas que la **normalización** evita:

- **Datos repetidos que se contradicen**: el país de Nubelia está dos veces, y ya dice `AR` en una y `Argentina` en otra. Para cambiarlo hay que tocar todas sus filas (*anomalía de actualización*).
- **No se puede guardar algo sin lo otro**: una empresa sin ofertas no tiene dónde ir (*anomalía de inserción*). Y si borrás la única oferta de una empresa, perdés la empresa (*anomalía de borrado*).

- **Listas en un texto**: buscar "las ofertas que piden TypeScript" con `LIKE '%TypeScript%'` es lento, no usa índices y confunde "Java" con "JavaScript".

Formas normales, en la práctica

Hay una teoría formal; para el día a día alcanza con tres reglas:

1. **Primera forma normal (1FN)**: cada celda tiene **un solo valor**. Nada de `'React, TypeScript'`: una lista es otra tabla.
2. **Segunda (2FN)**: en una tabla con clave compuesta, cada columna depende de **toda** la clave. En `oferta_tecnologia (oferta_id, tecnologia_id)` no va el nombre de la tecnología: depende solo de `tecnologia_id`.
3. **Tercera (3FN)**: cada columna depende **de la clave y de nada más**. `pais_empresa` en `ofertas` depende de la empresa, no de la oferta: va en `empresas`.

El resumen que se suele citar: cada dato vive **en un solo lugar**, y el resto lo referencia por su id.

Las relaciones y cómo se modelan

Uno a muchos (1:N). Una empresa tiene muchas ofertas. La clave foránea va del lado de los "muchos": `ofertas.empresa_id`.

Muchos a muchos (N:M). Una oferta pide muchas tecnologías y viceversa. Se modela con una **tabla intermedia** con las dos claves foráneas, y la combinación como clave primaria (así no se repite un par):

```
CREATE TABLE oferta_tecnologia (
  oferta_id      INT NOT NULL,
  tecnologia_id  INT NOT NULL,
  PRIMARY KEY (oferta_id, tecnologia_id),
  FOREIGN KEY (oferta_id) REFERENCES ofertas (id) ON DELETE CASCADE,
  FOREIGN KEY (tecnologia_id) REFERENCES tecnologias (id)
);
```

La tabla intermedia puede tener sus propias columnas (por ejemplo, `nivel` requerido, o `es_excluyente`).

Uno a uno (1:1). Cada oferta tiene como mucho una postulación. Es una clave foránea con `UNIQUE`:

SQL

```sql
-- Ya hay una postulación para la oferta 1
INSERT INTO postulaciones (oferta_id, fecha) VALUES (1, '2026-10-29');
```

RESULTADO EN MYSQL 8.0.46

```
ERROR 1062 (23000): Duplicate entry '1' for key
'postulaciones.oferta_id'
```

Qué pasa al borrar: ON DELETE

La clave foránea decide qué pasa con las filas "hijas" cuando se borra la "madre":

Opción	Al borrar la fila referenciada
`RESTRICT` / `NO ACTION` (**por defecto**)	Error: no se puede borrar mientras tenga hijas.
`CASCADE`	Se borran también las hijas.
`SET NULL`	En las hijas, la clave foránea pasa a `NULL` (la columna tiene que aceptar `NULL`).

En la base de ejemplo, `postulaciones.oferta_id` no dice nada (es `RESTRICT`) y `oferta_tecnologia.oferta_id` es `CASCADE`:

SQL

```sql
-- La oferta 1 tiene una postulación: RESTRICT no deja borrarla
DELETE FROM ofertas WHERE id = 1;
```

RESULTADO EN MYSQL 8.0.46

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key
constraint fails (`guia_sql`.`postulaciones`, CONSTRAINT
```

```
  `postulaciones_ibfk_1` FOREIGN KEY (`oferta_id`) REFERENCES `ofertas`
  (`id`))
```

SQL

```sql
-- La oferta 2 no tiene postulación: se borra, y sus tecnologías en
  cascada
SELECT COUNT(*) AS tecnologias_de_la_2 FROM oferta_tecnologia WHERE
  oferta_id = 2;
DELETE FROM ofertas WHERE id = 2;
SELECT COUNT(*) AS tecnologias_de_la_2 FROM oferta_tecnologia WHERE
  oferta_id = 2;
```

RESULTADO EN MYSQL 8.0.46

tecnologias_de_la_2
3

1 fila

```
Query OK, 1 fila afectada
```

tecnologias_de_la_2
0

1 fila

Elegí a propósito: `CASCADE` en lo que no tiene sentido sin la madre (las tecnologías de una oferta, los ítems de un pedido) y `RESTRICT` en lo que no querés perder por accidente (tus postulaciones, los pagos).

Elegir los tipos

Para	Usá	Evitá
Ids	`INT` o `BIGINT` `AUTO_INCREMENT` (o `BINARY(16)` para UUID)	`VARCHAR` como clave primaria
Plata	`DECIMAL(12, 2)` (o centavos en un `BIGINT`)	`FLOAT` / `DOUBLE`
Texto corto con límite	`VARCHAR(n)`	`TEXT` para todo
Texto largo	`TEXT`	

Para	Usá	Evitá
Fecha y hora	`DATETIME` (guardala en UTC) o `TIMESTAMP`	Fechas en `VARCHAR`
Sí/no	`BOOLEAN` (es un `TINYINT(1)`)	`'S'` / `'N'`
Conjunto fijo de valores	`ENUM`, o una tabla de referencia si puede crecer	
Texto en general	charset `utf8mb4` (el default de MySQL 8; incluye emojis)	`utf8` / `utf8mb3`, que no los tiene

¿Por qué no `FLOAT` para la plata? Porque guarda los números en binario y muchos decimales no tienen representación exacta:

SQL

```
SELECT CAST(0.1 AS DOUBLE) + CAST(0.2 AS DOUBLE) AS con_double,
       CAST(0.1 AS DECIMAL(10, 2)) + CAST(0.2 AS DECIMAL(10, 2)) AS
         con_decimal;
```

RESULTADO EN MYSQL 8.0.46

con_double	con_decimal
0.30000000000000004	0.30

1 fila

> 💡 **ENUM o tabla de referencia**
>
> `ENUM` es compacto y valida los valores, pero agregar uno nuevo es un `ALTER TABLE`, y ordena por la posición en la lista, no alfabéticamente. Si los valores cambian seguido o tienen datos propios (un nombre para mostrar, un color), usá una tabla aparte con una clave foránea.

Decisiones típicas

- **AUTO_INCREMENT o UUID.** El autoincremental es chico, rápido y ordenado, pero revela cuántos registros tenés (`/pedidos/1042`) y depende de la base para generarse. El UUID se puede generar en el cliente y no se adivina, pero ocupa más y, si es aleatorio, desordena

el índice de la clave primaria: si lo usás, que sea un UUID v7 (ordenado por tiempo) guardado en `BINARY(16)`.

- **Borrado lógico** (*soft delete*): en lugar de `DELETE`, una columna `eliminado_en DATETIME NULL`. Permite recuperar y auditar, pero cada consulta tiene que acordarse del `WHERE eliminado_en IS NULL`.
- **Desnormalizar a propósito.** A veces se duplica un dato para leer más rápido (un `cantidad_ofertas` en `empresas`, actualizado al insertar). Está bien si es una decisión medida y sabés quién lo mantiene sincronizado.
- **Columnas de auditoría.** `creado_en DATETIME DEFAULT CURRENT_TIMESTAMP` y `actualizado_en DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`, en casi todas las tablas.
- **Nombres.** Elegí una convención y mantenela: tablas en plural, columnas en `snake_case`, claves foráneas `<tabla_en_singular>_id`.

> 🎤 **En una entrevista**
>
> "Diseñá la base de un e-commerce." → Empezá por las entidades (usuarios, productos, categorías, pedidos, ítems del pedido, pagos) y las relaciones: un usuario tiene muchos pedidos (1:N), un pedido tiene muchos productos y un producto está en muchos pedidos (N:M, con la tabla `pedido_items` que guarda la cantidad y **el precio al momento de la compra**, porque el del producto cambia). Después: claves primarias, claves foráneas con su `ON DELETE`, `DECIMAL` para la plata, índices para las búsquedas frecuentes, y qué operaciones van en una transacción (crear el pedido y descontar el stock).

Capítulo 13

Vistas, procedimientos, triggers y JSON

Más herramientas de MySQL, usuarios y permisos, y backups.

Herramientas que no usás todos los días pero tenés que conocer: **vistas**, **procedimientos almacenados**, **triggers**, columnas **JSON**, y lo mínimo de administración (usuarios, permisos y backups).

Vistas

Una **vista** es una consulta guardada con nombre, que se usa como si fuera una tabla. No guarda datos: cada vez que la consultás, MySQL corre la consulta de adentro.

SQL

```sql
CREATE VIEW ofertas_detalle AS
SELECT o.id, o.titulo, o.seniority, o.modalidad, o.salario_max,
       e.nombre AS empresa, e.pais
FROM ofertas o
JOIN empresas e ON e.id = o.empresa_id;

SELECT titulo, empresa, salario_max
FROM ofertas_detalle
WHERE pais = 'PY' AND modalidad = 'remoto';
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas
```

titulo	empresa	salario_max
Fullstack Next.js	Tereré Tech	2800
Frontend React + TypeScript	Yerba Labs	3300

2 filas

Para qué sirven:

- **Esconder un join** que repetís en muchos lados.
- **Permisos**: le das a un usuario acceso a la vista (con algunas columnas) y no a la tabla entera.
- **Compatibilidad**: si cambiás las tablas, una vista con la forma vieja evita romper consultas existentes.

`CREATE OR REPLACE VIEW` la redefine y `DROP VIEW` la borra. Ojo con apilar vistas sobre vistas: el SQL final que ejecuta MySQL se vuelve difícil de seguir y de optimizar.

Procedimientos almacenados

Un **procedimiento** es código SQL guardado en el servidor, con parámetros, que se ejecuta con `CALL`:

SQL

```sql
CREATE PROCEDURE ofertas_remotas_desde(IN p_minimo INT)
  SELECT titulo, salario_max
  FROM ofertas
  WHERE modalidad = 'remoto' AND salario_max >= p_minimo
  ORDER BY salario_max DESC;

CALL ofertas_remotas_desde(4000);
```

RESULTADO EN MYSQL 8.0.46

Query OK, 0 filas afectadas

titulo	salario_max
Senior Frontend Engineer	6000
Tech Lead Frontend	5500
Frontend Engineer	4500
Frontend React Senior	4000

4 filas

Con más de una sentencia, el cuerpo va entre `BEGIN` y `END` y puede tener variables, `IF` y bucles. En la consola de `mysql` hay que cambiar el **delimitador** para que el `;` de adentro no termine el `CREATE`:

```
DELIMITER //
CREATE PROCEDURE pasar_a_entrevista(IN p_postulacion INT, IN p_fecha
  DATETIME)
BEGIN
  UPDATE postulaciones SET estado = 'entrevista' WHERE id =
    p_postulacion;
  INSERT INTO entrevistas (postulacion_id, fecha, tipo) VALUES
    (p_postulacion, p_fecha, 'rrhh');
END //
DELIMITER ;

CALL pasar_a_entrevista(4, '2026-11-03 10:00:00');
```

`DELIMITER` es un comando de la consola, no de SQL: desde Node, un cliente gráfico o una migración, el `CREATE PROCEDURE … BEGIN … END` se manda tal cual.

> ℹ️ **¿Lógica en la base o en la aplicación?**
>
> En empresas con mucho SQL (bancos, sistemas viejos) vas a encontrar mucha lógica en procedimientos. En aplicaciones modernas con Node, lo habitual es tener la lógica en el código de la aplicación (versionado, testeable, más fácil de leer) y usar la base para guardar y consultar. Los procedimientos se justifican cuando una operación mueve muchos datos y conviene no ir y volver por la red.

Triggers

Un **trigger** se ejecuta solo, **antes o después** de un `INSERT`, `UPDATE` o `DELETE` en una tabla. Adentro, `OLD` es la fila como estaba y `NEW` como queda. Un uso típico: un historial de cambios.

SQL

```
CREATE TABLE historial_estados (
  id              INT AUTO_INCREMENT PRIMARY KEY,
```

```
  postulacion_id   INT NOT NULL,
  estado_anterior  VARCHAR(20) NOT NULL,
  estado_nuevo     VARCHAR(20) NOT NULL,
  cambiado_en      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER postulaciones_historial
AFTER UPDATE ON postulaciones
FOR EACH ROW
  INSERT INTO historial_estados (postulacion_id, estado_anterior,
    estado_nuevo)
  VALUES (OLD.id, OLD.estado, NEW.estado);

UPDATE postulaciones SET estado = 'rechazado' WHERE id = 1;
UPDATE postulaciones SET estado = 'oferta' WHERE id = 5;

SELECT postulacion_id, estado_anterior, estado_nuevo
FROM historial_estados
ORDER BY id;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas
Query OK, 0 filas afectadas
Query OK, 1 fila afectada · Rows matched: 1 Changed: 1 Warnings: 0
Query OK, 1 fila afectada · Rows matched: 1 Changed: 1 Warnings: 0
```

postulacion_id	estado_anterior	estado_nuevo
1	entrevista	rechazado
5	entrevista	oferta

2 filas

> ⚠️ **Usalos con medida**
>
> Un trigger es lógica **invisible**: alguien hace un `UPDATE` y pasan cosas que no están en ningún lado del código de la aplicación. Son buenos para auditoría e historial. Para validar datos, primero mirá si alcanza con un `CHECK`, un `NOT NULL` o una clave foránea (MySQL respeta los `CHECK` desde la 8.0.16).

Columnas JSON

MySQL tiene un tipo `JSON`: valida que el contenido sea JSON y tiene funciones para leerlo. Sirve para datos con forma variable que no vale

la pena normalizar (preferencias, metadatos):

SQL

```sql
ALTER TABLE ofertas ADD COLUMN extra JSON NULL;

UPDATE ofertas
SET extra = '{"beneficios": ["prepaga", "equipo"], "zona_horaria": "UTC-3"}'
WHERE id = 1;

SELECT titulo,
       extra->>'$.zona_horaria' AS zona,
       JSON_LENGTH(extra->'$.beneficios') AS beneficios
FROM ofertas
WHERE extra IS NOT NULL;
```

RESULTADO EN MYSQL 8.0.46

```
Query OK, 0 filas afectadas · Records: 0 Duplicates: 0 Warnings: 0

Query OK, 1 fila afectada · Rows matched: 1 Changed: 1 Warnings: 0
```

titulo	zona	beneficios
Frontend React Senior	UTC-3	2

1 fila

`->` devuelve un valor JSON y `->>` lo devuelve como texto (sin comillas). Y al revés, se puede **armar JSON** desde una consulta, muy útil para devolver datos anidados a una API en una sola consulta:

SQL

```sql
SELECT e.nombre,
       JSON_ARRAYAGG(JSON_OBJECT('titulo', o.titulo, 'salario',
         o.salario_max)) AS ofertas
FROM empresas e
JOIN ofertas o ON o.empresa_id = e.id
WHERE e.pais = 'UY'
GROUP BY e.id, e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	ofertas
Pagos Sur	[{"titulo": "Frontend Engineer", "salario": 4500}, {"titulo": "React Native Developer", "salario": 3200}]

```
1 fila
```

No abuses: lo que filtrás, ordenás o unís seguido va en columnas normales. Un `WHERE` sobre un campo de un JSON no usa índices (salvo que crees una columna generada indexada).

Usuarios y permisos

Tu aplicación **no** se conecta como `root`. Se crea un usuario con los permisos justos (**principio de mínimo privilegio**): si alguien roba la contraseña o encuentra una inyección SQL, el daño queda limitado.

```
CREATE USER 'app_busqueda'@'%' IDENTIFIED BY 'una-clave-larga-y-
  aleatoria';

-- Solo leer y escribir datos de una base. Sin DROP, sin ALTER, sin otras
  bases.
GRANT SELECT, INSERT, UPDATE, DELETE ON busqueda.* TO 'app_busqueda'@'%';

SHOW GRANTS FOR 'app_busqueda'@'%';
REVOKE DELETE ON busqueda.* FROM 'app_busqueda'@'%';
DROP USER 'app_busqueda'@'%';
```

El `@'%'` es desde dónde se puede conectar: `'localhost'` solo desde la misma máquina, `'%'` desde cualquier lado. Las migraciones (que crean y alteran tablas) usan otro usuario con más permisos.

Backups

`mysqldump` exporta una base como un archivo `.sql` con los `CREATE TABLE` y los `INSERT`:

```
# Exportar (--single-transaction: una foto consistente sin bloquear las
  tablas InnoDB)
mysqldump -u root --single-transaction busqueda > busqueda-2026-10-30.sql

# Restaurar en otra base
mysql -u root -e "CREATE DATABASE busqueda_copia"
mysql -u root busqueda_copia < busqueda-2026-10-30.sql
```

Para bases grandes hay herramientas más rápidas (MySQL Shell con `util.dumpInstance`, o los snapshots del proveedor de la nube). Lo importante: un backup que nunca probaste restaurar no es un backup.

> **🎤 En una entrevista**
>
> "¿Qué es una vista y en qué se diferencia de una tabla?" → "Una consulta guardada con nombre que se usa como tabla, pero no guarda datos: se ejecuta cada vez. Sirve para reutilizar joins y para dar acceso a una parte de los datos sin dar acceso a la tabla. Si necesito los resultados guardados para leer rápido, eso es una vista materializada, que MySQL no tiene: se arma con una tabla que se actualiza con un job o un trigger."

Capítulo 14

MySQL desde Node

mysql2, pool, parámetros, SQL injection, transacciones en código, N+1 y ORMs.

Todo lo anterior, ahora desde una aplicación. El driver más usado para MySQL en Node es **mysql2** (con API de promesas). Este capítulo es la parte de SQL; el servidor con Express viene en la guía de Node.

```
npm install mysql2
```

Un pool de conexiones

Abrir una conexión a MySQL cuesta (red, autenticación). Un **pool** mantiene varias abiertas y las reparte entre las consultas que llegan. Se crea **uno** para toda la aplicación:

```
// db.ts
import mysql from 'mysql2/promise'

export const pool = mysql.createPool({
  host: process.env.DB_HOST ?? 'localhost',
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  connectionLimit: 10, // como mucho 10 conexiones a la vez
  timezone: 'Z', // fechas en UTC, sin sorpresas
})
```

Los datos de conexión van en **variables de entorno** (un `.env` que no se sube al repo), nunca en el código.

Consultas con parámetros

```ts
import type { RowDataPacket } from 'mysql2/promise'
import { pool } from './db'

interface Oferta extends RowDataPacket {
  id: number
  titulo: string
  salario_max: number | null
}

export async function ofertasRemotas(minimo: number) {
  const [filas] = await pool.execute<Oferta[]>(
    `SELECT id, titulo, salario_max
     FROM ofertas
     WHERE modalidad = 'remoto' AND salario_max >= ?
     ORDER BY salario_max DESC`,
    [minimo],
  )
  return filas // Oferta[]
}
```

- Cada `?` es un **parámetro**: el valor viaja **separado** del SQL, y MySQL nunca lo interpreta como código.
- `execute` usa *prepared statements* (MySQL compila la consulta una vez y la reusa); `query` reemplaza los `?` escapando los valores del lado del cliente. Las dos son seguras **si usás** `?`.
- Devuelve `[filas, campos]`. Para un `INSERT`, `UPDATE` o `DELETE`, el primer elemento es un `ResultSetHeader` con `insertId` y `affectedRows`.

```ts
import type { ResultSetHeader } from 'mysql2/promise'

const [resultado] = await pool.execute<ResultSetHeader>(
  'INSERT INTO postulaciones (oferta_id, fecha) VALUES (?, ?)',
  [ofertaId, '2026-10-30'],
)
console.log(resultado.insertId) // el id nuevo
```

SQL injection

El error de seguridad más famoso: **armar el SQL concatenando texto** que viene del usuario.

```
// ✗ NUNCA
const [filas] = await pool.query(`SELECT titulo FROM ofertas WHERE titulo
  = '${busqueda}'`)
```

Si alguien busca `' OR '1'='1`, la consulta que llega a MySQL es esta, y devuelve **todo**:

SQL

```
SELECT titulo FROM ofertas WHERE titulo = '' OR '1'='1';
```

RESULTADO EN MYSQL 8.0.46

titulo
Frontend React Senior
Fullstack Node
Frontend Engineer
React Native Developer
Desarrollador React
Frontend Junior
Tech Lead Frontend
Fullstack Next.js
Frontend React + TypeScript
Data Engineer
Frontend Senior
Senior Frontend Engineer
Backend Node.js
React Developer (LatAm)
Junior Frontend

15 filas

Y con un `UNION` puede leer **otras tablas** (por ejemplo, la de usuarios con sus contraseñas):

SQL

```
SELECT titulo FROM ofertas WHERE titulo = '' UNION SELECT CONCAT(nombre,
  ' · ', pais) FROM empresas -- ';
```

RESULTADO EN MYSQL 8.0.46

titulo
Nubelia · AR
Pagos Sur · UY
Andes Labs · CL
Mate Cloud · AR
Tereré Tech · PY
Yerba Labs · PY
Cumbre Data · MX
Delta Pay · BR
Remoto.dev · US
Quebracho AI · PY

10 filas

El `--` comenta la comilla que sobraba. Con parámetros, ese texto es solo un título raro que no coincide con nada:

```
// ✓ Siempre
const [filas] = await pool.execute('SELECT titulo FROM ofertas WHERE
  titulo = ?', [busqueda])
```

> ⚠️ **Lo que no puede ir en un ?**
>
> Los parámetros son para **valores**, no para nombres de columnas o tablas, ni para `ASC` / `DESC`. Si el usuario elige por qué columna ordenar, validala contra una lista fija:
>
> ```
> const columnas = { salario: 'salario_max', fecha: 'publicada' } as
>   const
> const columna = columnas[req.query.orden as keyof typeof columnas]
>   ?? 'publicada'
> const sql = `SELECT titulo FROM ofertas ORDER BY ${columna} DESC
>   LIMIT ?`
> ```
>
> Para una lista con `IN`, `query` expande un array: `pool.query('… WHERE id IN (?)', [[1, 2, 3]])`.

Transacciones desde el código

Una transacción tiene que ir en **una misma conexión**, así que se pide una al pool:

```
export async function aceptarOferta(postulacionId: number) {
  const conexion = await pool.getConnection()
  try {
    await conexion.beginTransaction()
    await conexion.execute("UPDATE postulaciones SET estado = 'oferta'
      WHERE id = ?", [postulacionId])
    await conexion.execute(
      "UPDATE entrevistas SET resultado = 'paso' WHERE postulacion_id = ?
        AND resultado = 'pendiente'",
      [postulacionId],
    )
    await conexion.commit()
  } catch (error) {
    await conexion.rollback() // si algo falló, se deshace todo
    throw error
  } finally {
    conexion.release() // la conexión vuelve al pool, pase lo que pase
  }
}
```

Si te olvidás del `release()`, el pool se queda sin conexiones y la aplicación se cuelga esperando.

El problema N+1

Un clásico en el código de una API: traer una lista y después, **por cada elemento**, hacer otra consulta.

```
// ✗ 1 consulta para las empresas + 1 por cada empresa = N+1 consultas
const [empresas] = await pool.query<Empresa[]>('SELECT id, nombre FROM
  empresas')
for (const empresa of empresas) {
  const [ofertas] = await pool.execute<Oferta[]>('SELECT titulo FROM
    ofertas WHERE empresa_id = ?', [empresa.id])
  empresa.ofertas = ofertas
}
```

Con 10 empresas son 11 viajes a la base; con 1000, 1001. Se resuelve con **una** consulta con `JOIN` (y armando la estructura en JavaScript), o con dos: las empresas, y después todas sus ofertas con `WHERE empresa_id IN (?)`. También se puede pedir el JSON armado a MySQL, como en el capítulo de vistas y JSON.

Lo que devuelve mysql2

- `DATE` y `DATETIME` llegan como objetos `Date` (con `dateStrings: true`, como texto).
- `DECIMAL` y `BIGINT` llegan como **string**, para no perder precisión en JavaScript. Convertilos a propósito (o usá las opciones `decimalNumbers` y `supportBigNumbers`).
- `TINYINT(1)` (un `BOOLEAN`) llega como `0` o `1`, no como `true` / `false`.
- `JSON` llega ya parseado.

ORMs y query builders

En muchos proyectos no vas a escribir SQL a mano sino con una librería:

Herramienta	Qué es	Cómo se siente
Prisma	ORM con su propio archivo de esquema y un cliente generado, muy tipado	`prisma.oferta.findMany({ where: { modalidad: 'remoto' }, include: { empresa: true } })`
Drizzle	Query builder tipado, el esquema se escribe en TypeScript	`db.select().from(ofertas).where(eq(ofertas.modalidad, 'remoto'))`
Knex	Query builder clásico, sin tipos fuertes	`knex('ofertas').where({ modalidad: 'remoto' })`
Sequelize / **TypeORM**	ORMs con modelos y clases (más comunes en código legacy)	`Oferta.findAll({ where: { modalidad: 'remoto' } })`

Todos usan parámetros por dentro (te protegen de la inyección), generan las **migraciones** (los cambios de esquema versionados, como commits de la base) y tipan los resultados. Pero el SQL sigue ahí: cuando una consulta es lenta, vas a mirar el SQL que generan y su `EXPLAIN`. Por eso conviene saber lo de esta guía aunque uses un ORM.

🎤 En una entrevista

"¿Cómo evitás el SQL injection?" → "Nunca armo SQL concatenando datos del usuario: uso consultas con parámetros (los ? de mysql2, o un ORM o query builder que los usa por dentro), así el valor viaja separado del SQL y no se interpreta como código. Lo que no puede ser parámetro, como el nombre de una columna para ordenar, lo valido contra una lista fija. Y además, la aplicación se conecta con un usuario de MySQL con los permisos mínimos."

Parte 3 · Repaso

Todo junto para repasar antes de una entrevista.

Capítulo 15

Chuleta y ejercicios de entrevista

La sintaxis en una página, ejercicios resueltos y preguntas frecuentes.

Todo en una página, para repasar antes de una entrevista o tener a mano mientras escribís SQL.

El orden en que se ejecuta un SELECT

Se escribe en un orden y se **evalúa** en otro. Saberlo explica la mitad de los errores:

Paso	Cláusula	Por eso...
1	`FROM` y `JOIN`	se arma primero la tabla combinada
2	`WHERE`	filtra filas: no puede usar agregaciones ni funciones de ventana
3	`GROUP BY`	agrupa
4	`HAVING`	filtra grupos: sí puede usar `COUNT`, `SUM`...
5	Funciones de ventana	se calculan sobre lo que quedó
6	`SELECT`	recién acá existen los alias de columna
7	`DISTINCT`	
8	`ORDER BY`	puede usar los alias del `SELECT`
9	`LIMIT` / `OFFSET`	

Tablas

```sql
CREATE TABLE ofertas (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  empresa_id  INT NOT NULL,
  titulo      VARCHAR(150) NOT NULL,
  salario     DECIMAL(10, 2) NULL CHECK (salario > 0),
  modalidad   ENUM('remoto', 'hibrido', 'presencial') NOT NULL DEFAULT
    'remoto',
  creado_en   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (empresa_id, titulo),
  FOREIGN KEY (empresa_id) REFERENCES empresas (id) ON DELETE CASCADE
);

ALTER TABLE ofertas ADD COLUMN url VARCHAR(300) NULL;
ALTER TABLE ofertas MODIFY titulo VARCHAR(200) NOT NULL;
ALTER TABLE ofertas DROP COLUMN url;
ALTER TABLE ofertas RENAME COLUMN salario TO salario_max;
DROP TABLE IF EXISTS ofertas;
TRUNCATE TABLE ofertas;       -- vacía la tabla y reinicia el
  AUTO_INCREMENT

SHOW TABLES;
DESCRIBE ofertas;
SHOW CREATE TABLE ofertas;
```

Escribir datos

```sql
INSERT INTO empresas (nombre, pais) VALUES ('Nubelia', 'AR'), ('Pagos
  Sur', 'UY');
INSERT INTO resumen (pais, total) SELECT pais, COUNT(*) FROM empresas
  GROUP BY pais;

UPDATE ofertas SET salario_max = salario_max * 1.1 WHERE empresa_id = 3;
DELETE FROM ofertas WHERE publicada < '2026-01-01';

-- Upsert: insertar o, si choca con una clave única, actualizar
INSERT INTO empresas (nombre, pais, empleados) VALUES ('Nubelia', 'AR',
  150) AS nueva
ON DUPLICATE KEY UPDATE empleados = nueva.empleados;
```

`UPDATE` y `DELETE` **sin** `WHERE` cambian toda la tabla. Antes de correrlos, probá el `WHERE` con un `SELECT`.

Consultar

```sql
SELECT e.pais, COUNT(*) AS ofertas, AVG(o.salario_max) AS promedio
FROM ofertas o
JOIN empresas e ON e.id = o.empresa_id          -- solo las que tienen
  pareja
LEFT JOIN postulaciones p ON p.oferta_id = o.id  -- todas las de la
  izquierda
WHERE o.modalidad = 'remoto'
  AND o.publicada BETWEEN '2026-10-01' AND '2026-10-31'
  AND o.titulo LIKE '%React%'
  AND e.pais IN ('AR', 'PY')
  AND p.id IS NULL                               -- sin postulación
    (anti-join)
GROUP BY e.pais
HAVING COUNT(*) >= 2
ORDER BY ofertas DESC, e.pais
LIMIT 10 OFFSET 20;
```

Subconsultas, CTE y ventanas

```sql
WHERE salario_max > (SELECT AVG(salario_max) FROM ofertas)        -- un
  valor
WHERE empresa_id IN (SELECT id FROM empresas WHERE pais = 'PY')    -- una
  lista
WHERE NOT EXISTS (SELECT 1 FROM postulaciones p WHERE p.oferta_id = o.id)

WITH remotas AS (SELECT * FROM ofertas WHERE modalidad = 'remoto'),
     promedio AS (SELECT AVG(salario_max) AS valor FROM remotas)
SELECT ... FROM remotas, promedio;

WITH RECURSIVE n AS (SELECT 1 AS i UNION ALL SELECT i + 1 FROM n WHERE i
  < 10)
SELECT i FROM n;

-- El mejor de cada grupo
WITH r AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY seniority ORDER BY
    salario_max DESC) AS puesto
  FROM ofertas
)
SELECT * FROM r WHERE puesto = 1;

SUM(x) OVER (ORDER BY fecha)                          -- acumulado
AVG(x) OVER (PARTITION BY grupo)                      -- promedio del
  grupo en cada fila
RANK() / DENSE_RANK() OVER (ORDER BY x DESC)          -- con empates
```

```
LAG(x) OVER (ORDER BY fecha), LEAD(x) OVER (...)     -- fila anterior y
  siguiente
```

Funciones útiles

Para	Funciones
Texto	`CONCAT`, `CONCAT_WS`, `UPPER`, `LOWER`, `TRIM`, `SUBSTRING`, `REPLACE`, `CHAR_LENGTH`, `LPAD`
Números	`ROUND`, `FLOOR`, `CEIL`, `ABS`, `MOD`, `GREATEST`, `LEAST`
Fechas	`NOW()`, `CURDATE()`, `DATE()`, `YEAR()`, `MONTH()`, `DATEDIFF(a, b)`, `DATE_ADD(f, INTERVAL 7 DAY)`, `f + INTERVAL 1 MONTH`, `DATE_FORMAT(f, '%d/%m/%Y')`, `TIMESTAMPDIFF(HOUR, a, b)`
NULL	`IS NULL`, `COALESCE(a, b, …)`, `IFNULL(a, b)`, `NULLIF(a, b)`, `<=>`
Condiciones	`CASE WHEN … THEN … ELSE … END`, `IF(cond, a, b)`
Agregación	`COUNT(*)`, `COUNT(col)`, `COUNT(DISTINCT col)`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP_CONCAT(col ORDER BY col SEPARATOR ', ')`
JSON	`JSON_OBJECT`, `JSON_ARRAYAGG`, `col->'$.campo'`, `col->>'$.campo'`, `JSON_LENGTH`

Índices y planes

```
CREATE INDEX idx_empresa_fecha ON ofertas (empresa_id, publicada);  --
  igualdad primero, rango después
CREATE UNIQUE INDEX idx_email ON usuarios (email);
DROP INDEX idx_empresa_fecha ON ofertas;
SHOW INDEX FROM ofertas;

EXPLAIN SELECT ...;           -- el plan: mirá type, key, rows y Extra
EXPLAIN ANALYZE SELECT ...;   -- lo ejecuta y muestra tiempos reales
```

Rompen un índice: funciones sobre la columna (`YEAR(fecha) = 2026`), `LIKE '%texto'` , operaciones (`col * 12 > x`), saltearse la primera columna de un índice compuesto.

Transacciones y permisos

```
START TRANSACTION;
```

```sql
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;                -- o ROLLBACK;

SELECT ... FOR UPDATE; -- bloquea las filas leídas hasta el COMMIT

CREATE USER 'app'@'%' IDENTIFIED BY '...';
GRANT SELECT, INSERT, UPDATE, DELETE ON busqueda.* TO 'app'@'%';
```

Ejercicios de entrevista

Problemas típicos de una prueba técnica, resueltos sobre la base de la guía.

Las 3 empresas con más ofertas remotas.

SQL

```sql
SELECT e.nombre, COUNT(*) AS remotas
FROM empresas e
JOIN ofertas o ON o.empresa_id = e.id
WHERE o.modalidad = 'remoto'
GROUP BY e.id, e.nombre
ORDER BY remotas DESC, e.nombre
LIMIT 3;
```

RESULTADO EN MYSQL 8.0.46

nombre	remotas
Delta Pay	2
Nubelia	2
Remoto.dev	2

3 filas

Por cada tecnología: cuántas ofertas la piden y el mejor salario. Incluí las que nadie pide.

SQL

```sql
SELECT t.nombre,
       COUNT(o.id) AS ofertas,
       MAX(o.salario_max) AS mejor_salario
```

```
FROM tecnologias t
LEFT JOIN oferta_tecnologia ot ON ot.tecnologia_id = t.id
LEFT JOIN ofertas o ON o.id = ot.oferta_id
GROUP BY t.id, t.nombre
ORDER BY ofertas DESC, t.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre	ofertas	mejor_salario
React	11	6000
TypeScript	8	6000
Next.js	4	6000
Node.js	4	3500
AWS	3	5500
Redux	3	4000
MySQL	2	3000
PostgreSQL	2	4200
Python	1	4200
Vue	0	*NULL*

10 filas

La oferta más reciente de cada empresa.

SQL

```
WITH ordenadas AS (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY empresa_id ORDER BY
    publicada DESC) AS n
  FROM ofertas o
)
SELECT e.nombre, ord.titulo, ord.publicada
FROM ordenadas ord
JOIN empresas e ON e.id = ord.empresa_id
WHERE ord.n = 1
ORDER BY ord.publicada DESC;
```

RESULTADO EN MYSQL 8.0.46

nombre	titulo	publicada
Remoto.dev	Junior Frontend	2026-10-25
Delta Pay	Backend Node.js	2026-10-22
Yerba Labs	Frontend React + TypeScript	2026-10-18
Mate Cloud	Tech Lead Frontend	2026-10-12
Pagos Sur	React Native Developer	2026-10-05
Tereré Tech	Fullstack Next.js	2026-10-03
Cumbre Data	Frontend Senior	2026-09-25

nombre	titulo	publicada
Nubelia	Fullstack Node	2026-09-20
Andes Labs	Desarrollador React	2026-09-15

9 filas

Las empresas a las que no te postulaste nunca.

SQL

```sql
SELECT e.nombre
FROM empresas e
WHERE NOT EXISTS (
  SELECT 1
  FROM ofertas o
  JOIN postulaciones p ON p.oferta_id = o.id
  WHERE o.empresa_id = e.id
)
ORDER BY e.nombre;
```

RESULTADO EN MYSQL 8.0.46

nombre
Cumbre Data
Quebracho AI

2 filas

El embudo: de tus postulaciones, qué porcentaje llegó a entrevista o más.

SQL

```sql
SELECT COUNT(*) AS postulaciones,
       SUM(estado IN ('entrevista', 'oferta')) AS avanzaron,
       ROUND(100 * SUM(estado IN ('entrevista', 'oferta')) / COUNT(*), 1)
         AS porcentaje
FROM postulaciones;
```

RESULTADO EN MYSQL 8.0.46

postulaciones	avanzaron	porcentaje
8	4	50.0

1 fila

En MySQL una condición vale 1 o 0, así que `SUM(condición)` cuenta las filas que la cumplen. En otras bases se escribe `SUM(CASE WHEN … THEN 1 ELSE 0 END)`.

Los países con más de una empresa, y cuáles son.

SQL

```sql
SELECT pais,
       COUNT(*) AS empresas,
       GROUP_CONCAT(nombre ORDER BY nombre SEPARATOR ', ') AS nombres
FROM empresas
GROUP BY pais
HAVING COUNT(*) > 1
ORDER BY empresas DESC, pais;
```

RESULTADO EN MYSQL 8.0.46

pais	empresas	nombres
PY	3	Quebracho AI, Tereré Tech, Yerba Labs
AR	2	Mate Cloud, Nubelia

2 filas

Preguntas rápidas

🎤 Preguntas frecuentes

- **¿Clave primaria vs clave foránea?** La primaria identifica cada fila (única, no nula, una por tabla). La foránea guarda la primaria de otra tabla y MySQL verifica que exista.
- **¿WHERE vs HAVING?** `WHERE` filtra filas antes de agrupar; `HAVING` filtra grupos y puede usar agregaciones.
- **¿INNER vs LEFT JOIN?** `INNER` solo devuelve filas con pareja; `LEFT` devuelve todas las de la izquierda, con `NULL` si no hay pareja.
- **¿DELETE vs TRUNCATE vs DROP?** `DELETE` borra filas (con `WHERE`, dentro de una transacción); `TRUNCATE` vacía la tabla de una (más rápido, reinicia el `AUTO_INCREMENT`, no se puede deshacer); `DROP` borra la tabla entera.
- **¿UNION vs UNION ALL?** `UNION` elimina duplicados (más lento); `UNION ALL` los deja.
- **¿Por qué `= NULL` no funciona?** Cualquier comparación con `NULL` da `NULL` (desconocido), no verdadero. Se usa `IS NULL`.
- **¿Qué es un índice y cuándo no conviene?** Una estructura ordenada (árbol B+) que evita leer toda la tabla. Hace más lentas las escrituras y ocupa espacio; no sirve en columnas con pocos valores distintos.
- **¿Qué es ACID?** Atomicidad, Consistencia, Aislamiento y Durabilidad: las garantías de una transacción.
- **¿Qué es la normalización?** Organizar las tablas para que cada dato viva en un solo lugar y evitar inconsistencias (1FN: valores atómicos; 2FN y 3FN: cada columna depende de toda la clave y solo de la clave).
- **¿Qué es el SQL injection?** Meter código SQL a través de un dato del usuario que se concatena en la consulta. Se evita con consultas con parámetros.
- **¿Qué es el problema N+1?** Hacer una consulta por cada elemento de una lista. Se resuelve con un `JOIN` o con `IN`.
- **¿SQL o NoSQL?** SQL cuando los datos tienen relaciones y necesitás consistencia y transacciones (la mayoría de las apps). NoSQL (MongoDB, DynamoDB) cuando los datos son

documentos sin relaciones fuertes o se necesita escalar escrituras horizontalmente.